

Robot Learning

General course information
Basics of robotics
Fundamentals of machine learning

Team



Prof. Erdem Bıyık
Instructor
biyik@usc.edu

Office hours

- Erdem:
 - Wednesday, 2:00pm – 3:00pm, GCS SB3 (Floor: LL2)

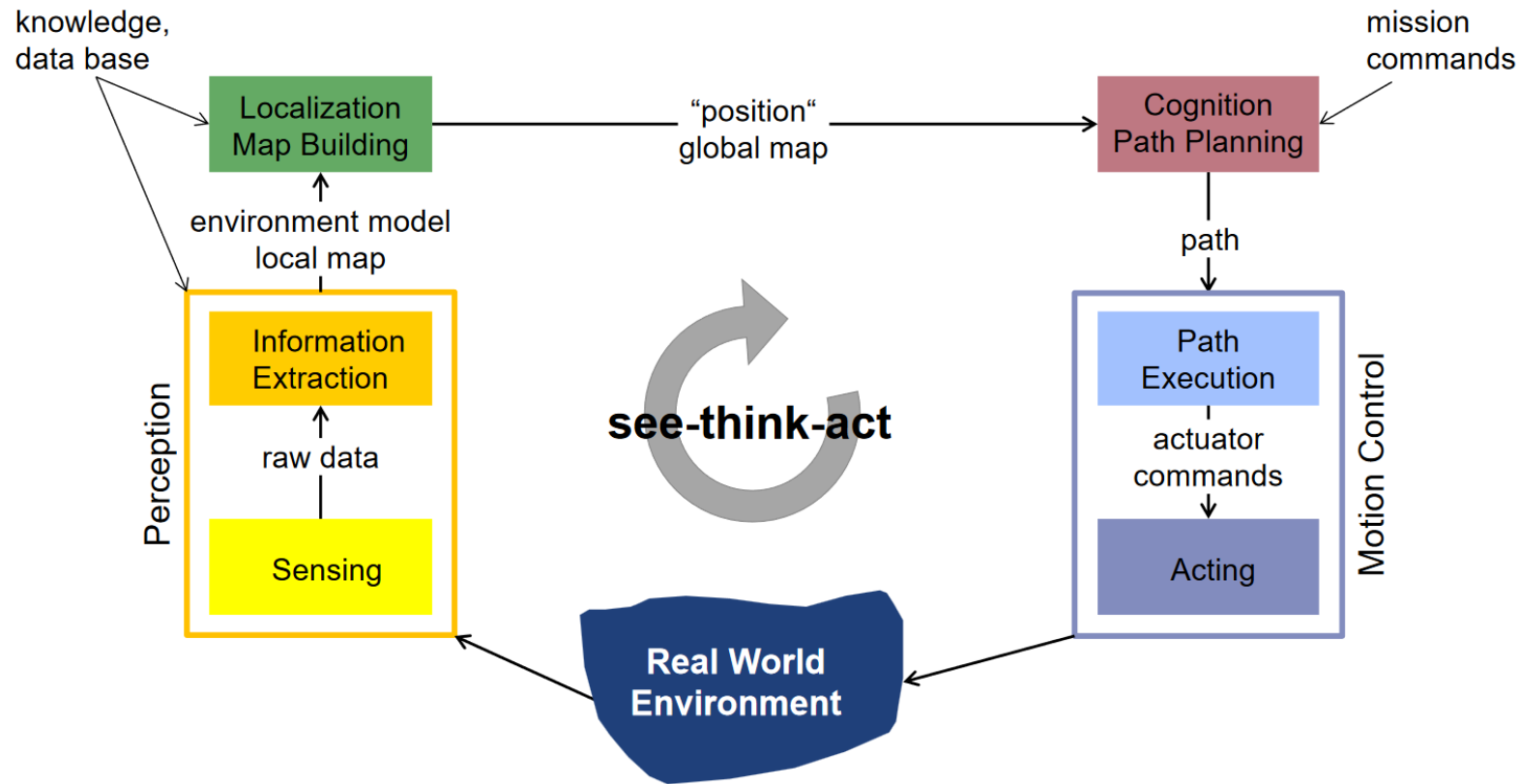
Online resources

- Course website: <https://liralab.usc.edu/csci642/>
- Piazza: <https://piazza.com/usc/fall2026/csci642>
- Gradescope: <https://www.gradescope.com/courses/1370900>

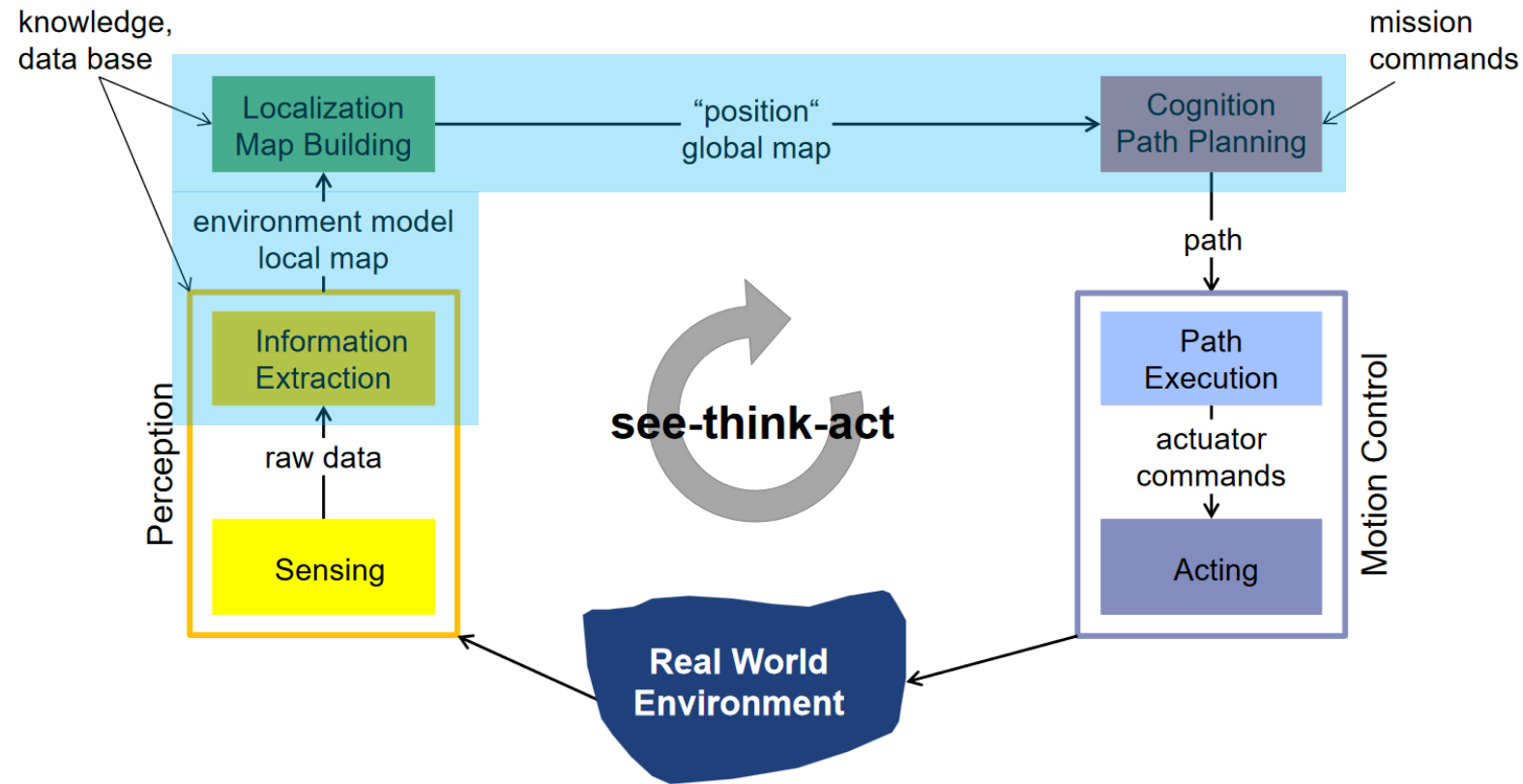
What is a robot?

- An embodied artificial intelligence
- A machine that can autonomously carry out useful work
- An artificial device that can sense its environment and purposefully act on or in that environment

See-think-act cycle



Robot learning



Why robot learning?

Designing controllers is hard

- Requires good understanding of the system
- Doesn't scale well to high-dimensional systems
- *“Manipulation breaks all the rigorous/reliable approaches I know for control.”*
 - Russ Tedrake (MIT / TRI)

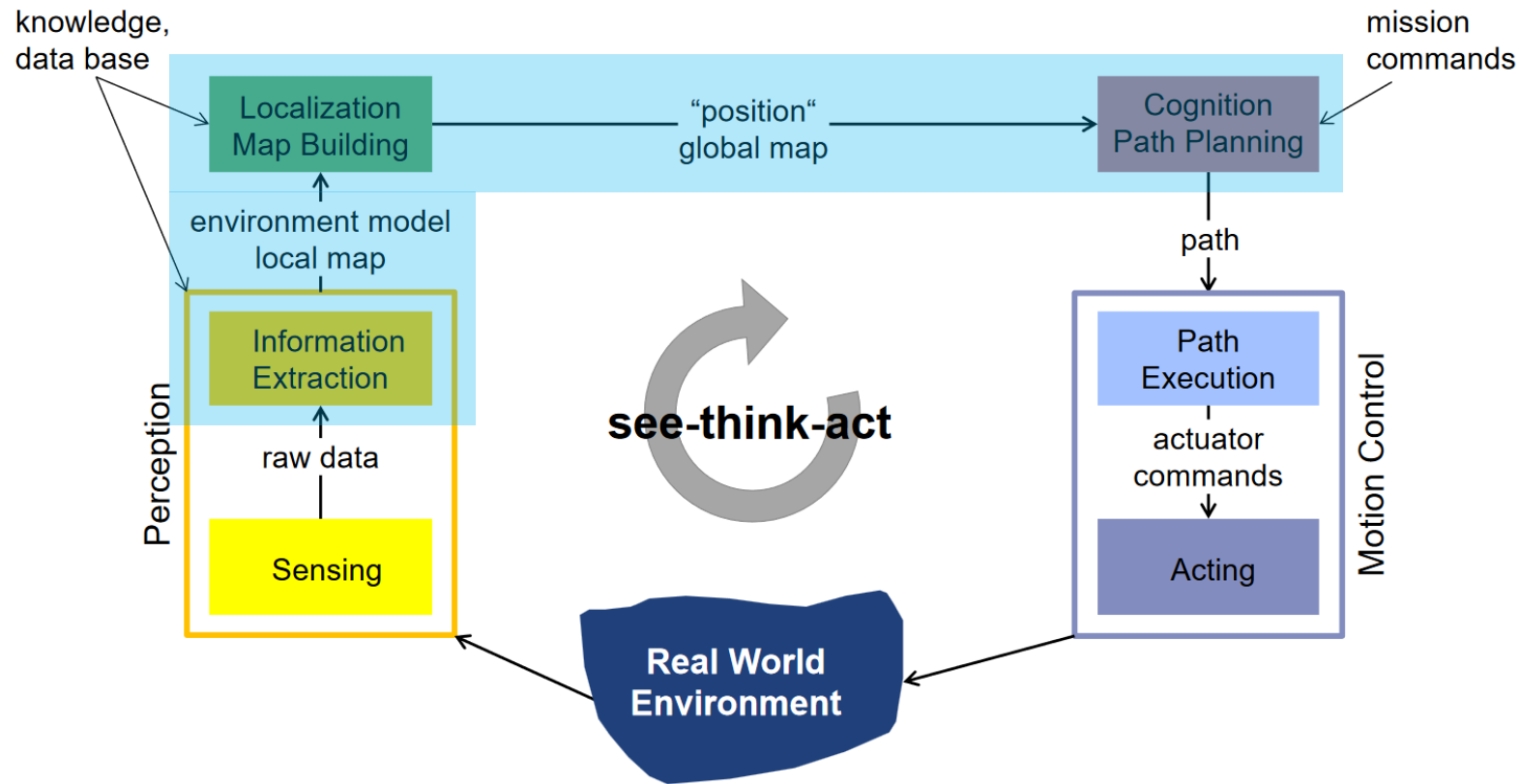
Prerequisites

- Probability theory
- Calculus
- Linear algebra
- At least one programming language (preferably, Python)
 - Programming assignments will be in Python.
- Recommended:
 - Familiarity with basic concepts in machine learning

What's covered?

- Basics of...
 - Robotics
 - Machine learning
 - Computer vision
- Representation learning
- Reinforcement learning
- Imitation learning / IRL
- Learning from human feedback
- Sim-to-real transfer
- Meta-learning
- Safe and robust learning
- Multi-agent learning
- Robot learning using natural language

What's NOT covered?



What's NOT covered?

- Robot operating system (ROS)
 - CSCI 545: Robotics
- Simultaneous localization and mapping (SLAM)
 - CSCI 545: Robotics
- Grasping and manipulation
 - CSCI 645: Deep Learning for Robotic Manipulation
- Haptics
 - CSCI 649: Haptic Interfaces and Virtual Environments

Textbook & Readings

- No textbook is required.
- All readings will be available on course website.
- If I were to recommend textbooks for this class...
 - Reinforcement Learning: An Introduction by Sutton and Barto
 - Modern Adaptive Control and Reinforcement Learning (MACRL) by Bagnell, Boots, and Choudhury
 - Principles of Robot Autonomy by Lorenzetti and Pavone

Assignments

- Readings (7%)
- Two homework assignments (2 x 10%)
- One paper presentation (15%)
- One midterm exam (15%)
- Course project (38%)
- Peer review (5%)

Homework assignments

- Three homework assignments in total – you choose which two you would like to do!
 - Basics of robotics & machine learning & computer vision
 - Reinforcement learning
 - Imitation learning & intent inference & shared autonomy
- Both theoretical and programming components
- Programming parts will be in Python
- No ROS knowledge required
- The submissions will be online, due at 12 midnight.

Homework assignments

- For now, the class does not have a TA or a grader.
- They told us to be creative about how to handle grading.
- After every deadline, I will share the solution key with you.
- Everyone grades themselves in two weeks.
- I randomly pick n submissions and grade them myself.
 - If I realize the student cheated by increasing their score by x , I give a penalty of kx/n points where k is the number of students who submitted the homework.
- I continue grading until I find n fairly graded assignments.

Class presentation

- 20-30 minutes presentation, depending on the week/paper
- Should include an extensive discussion of the paper
 - Motivation
 - Prior work
 - Methods
 - Results
 - Discussion
 - Both the positive and the negative aspects of the paper!
- 5 minutes Q&A

Course project

- The project will be done in groups of 2 or 3.
- Feel free to reach out to me if you have a good reason to do it individually or as a group of more than 3 students.

Course project

The project must have both robotics and machine learning components.

Examples:

- Application-dependent improvements over an existing robot learning method
- A new application of an existing robot learning technique
- A novel method that **may** have potential benefits

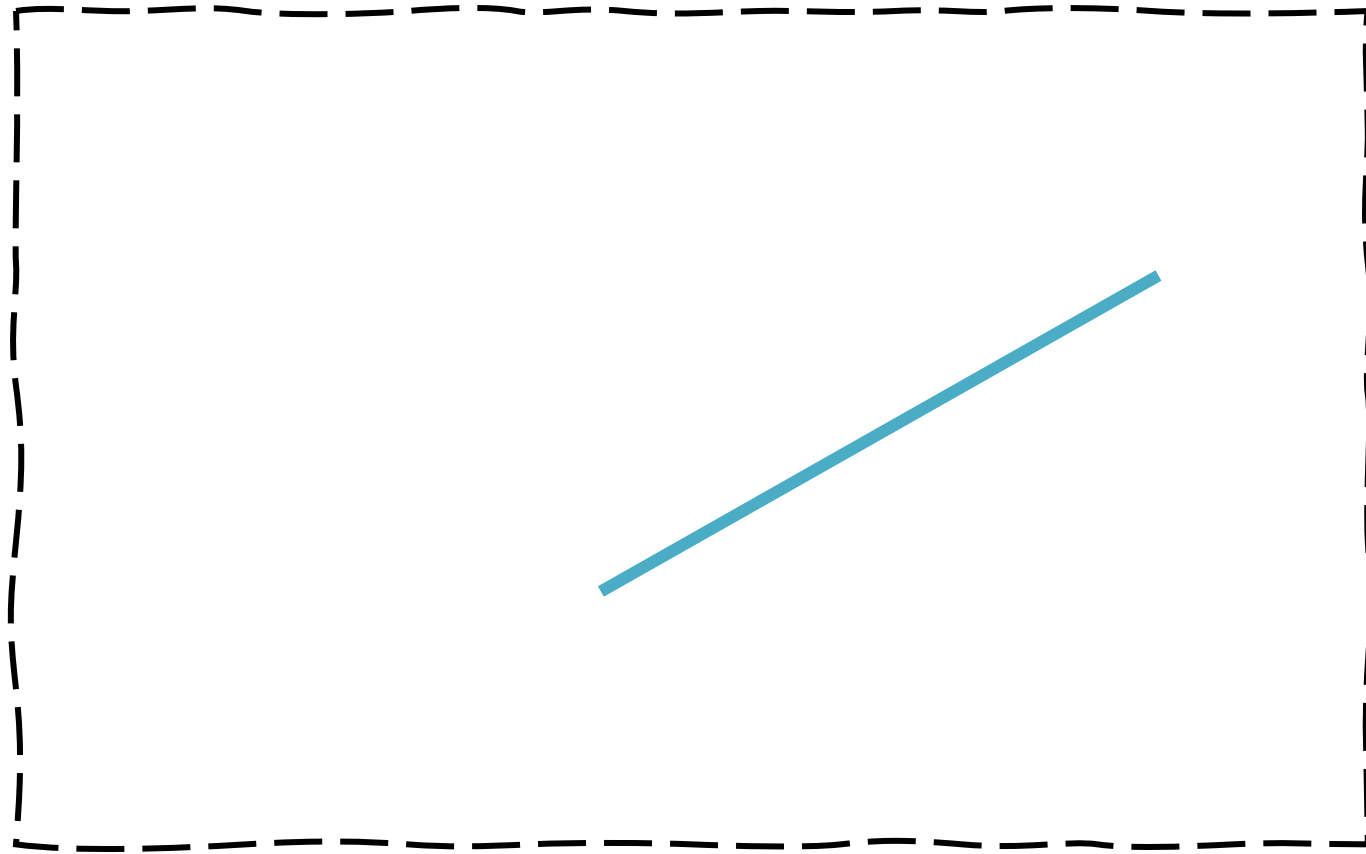
Course project

Component	Contribution to Grade	
Project Proposal Report	5%	October 14 th
Project Milestone Report	8%	November 4 th
Project Presentation (Possibly with Demo)	10%	December 2 th
Final Project Report	15%	December 4 th
Peer Review	5%	December 11 th
Total	43%	

Today...

- General course information
- Basics of robotics
- Fundamentals of machine learning

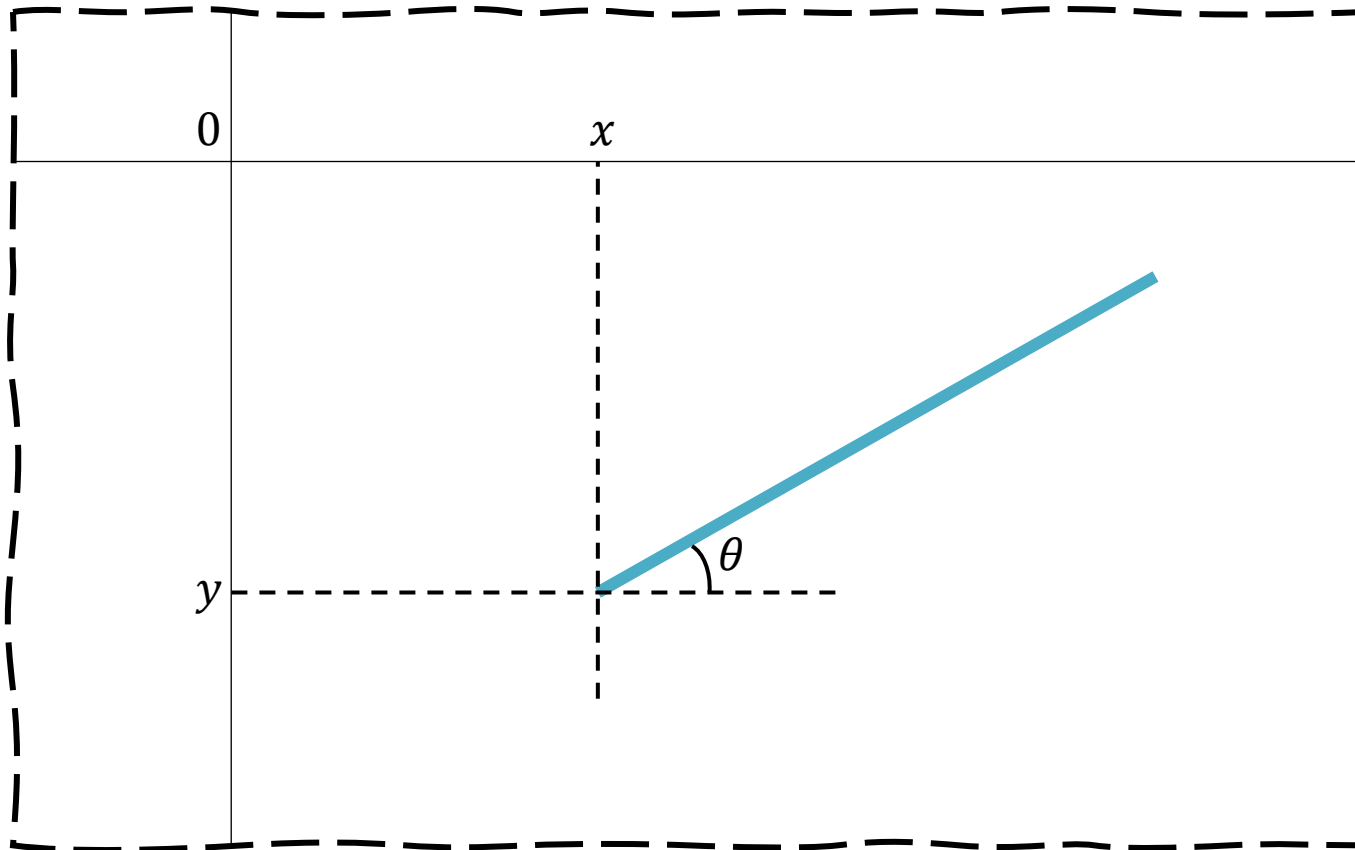
A rigid body in 2D space



This rigid body is free to move and rotate in any direction.

How many variables do we need to fully describe the configuration of this rigid body?

A rigid body in 2D space

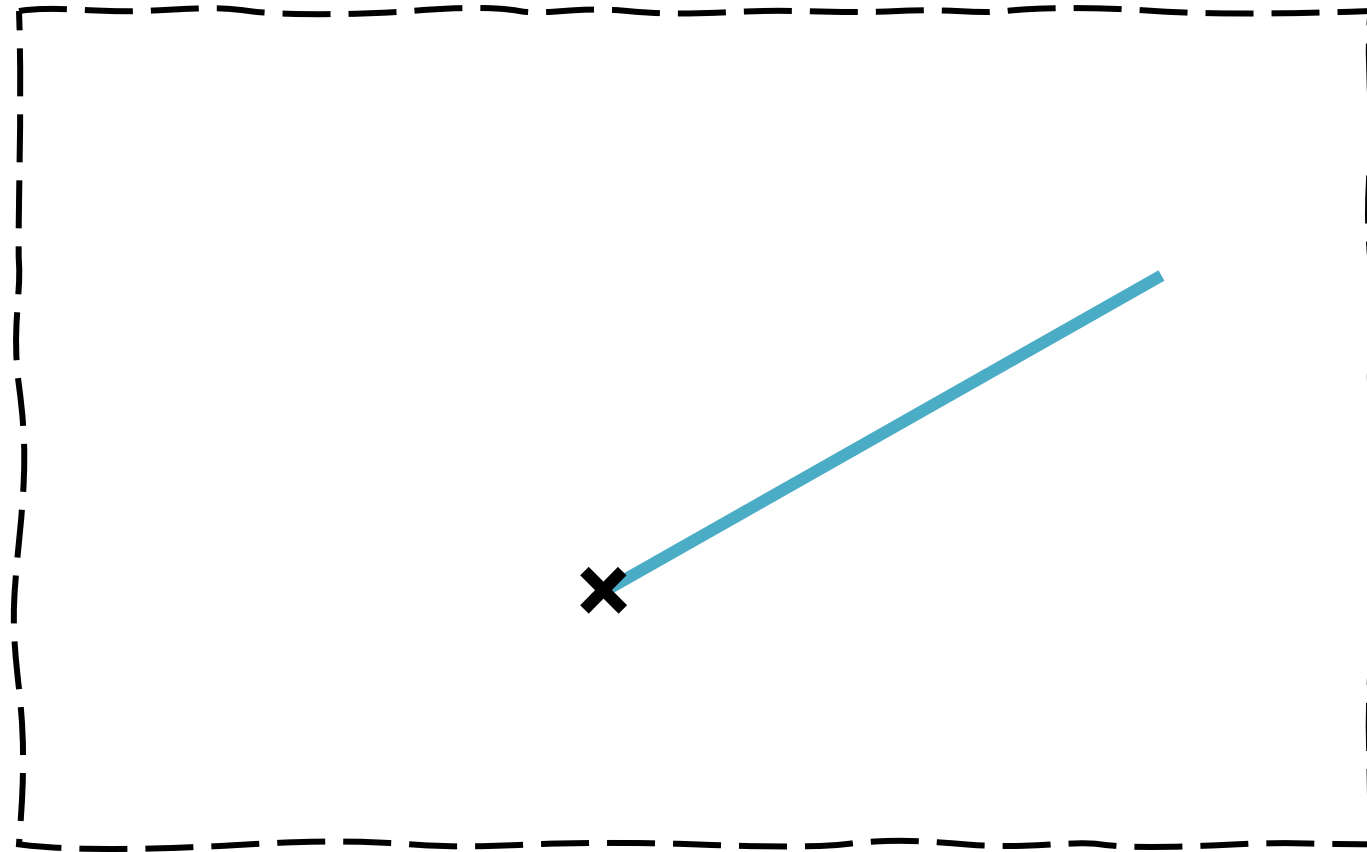


This rigid body is free to move and rotate in any direction.

How many variables do we need to fully describe the configuration of this rigid body?

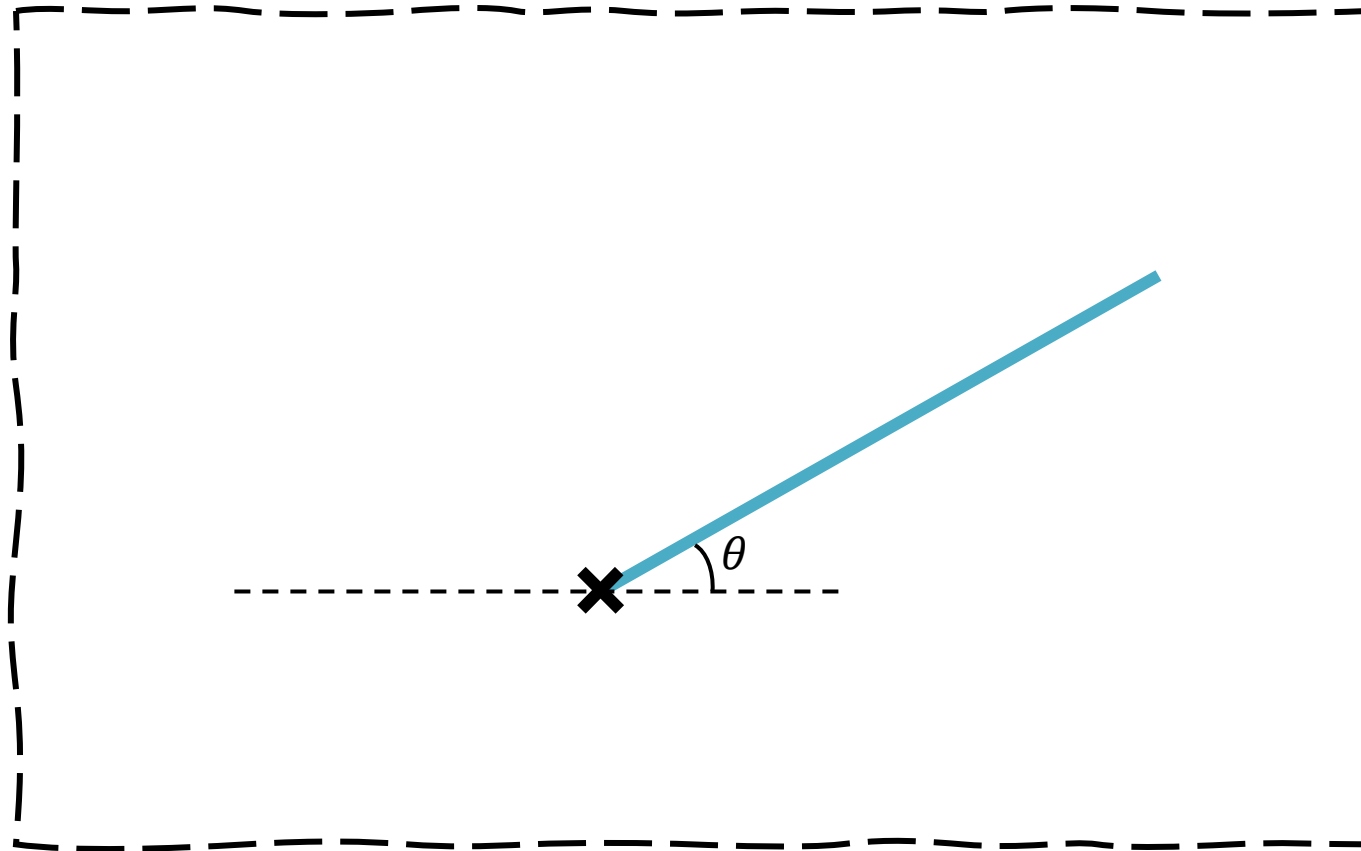
The answer is 3 variables: (x, y, θ)

A rigid body in 2D space



What if one of the end points is fixed?

A rigid body in 2D space



What if one of the end points is fixed?

Two of the variables are now fixed by two constraints:

$$x = \bar{x}$$

$$y = \bar{y}$$

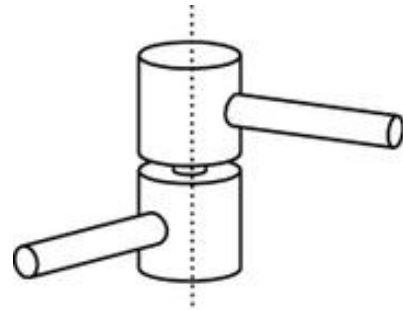
We only need one variable: θ

This is called the **degree-of-freedom (DoF)** of the body.

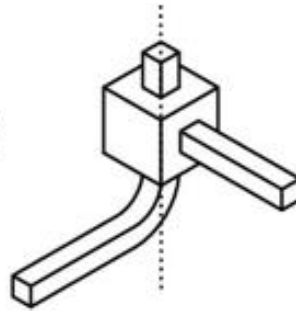
A rigid body in 3D space

- Requires 6 degrees of freedom:
 - Three for position
 - Three for orientation

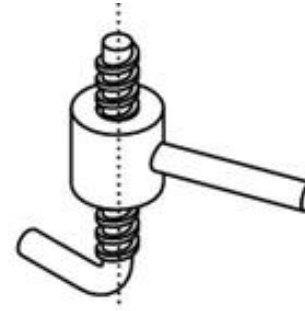
Common joints



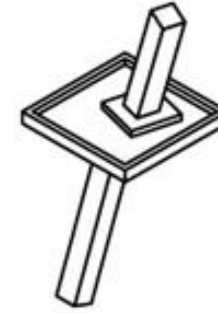
Revolute joint



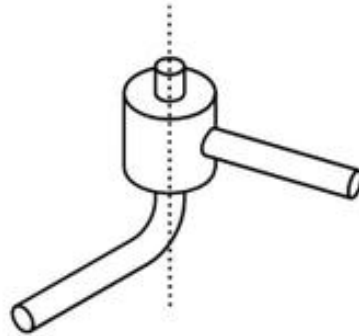
Prismatic joint



Helical joint



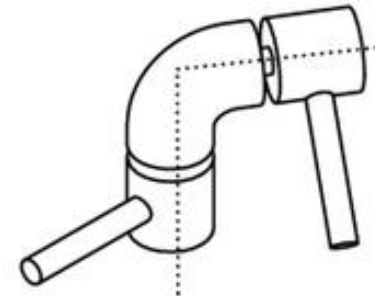
Planar joint



Cylindrical joint

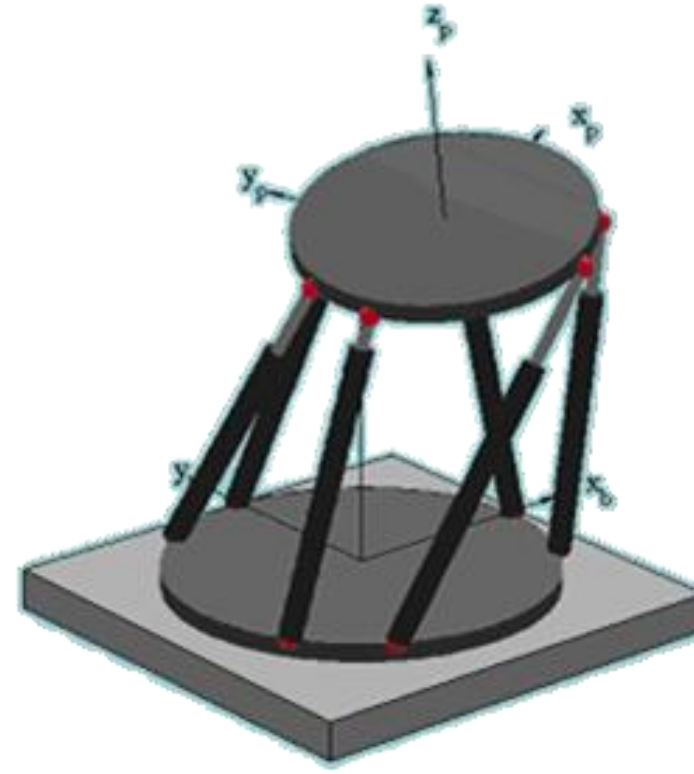
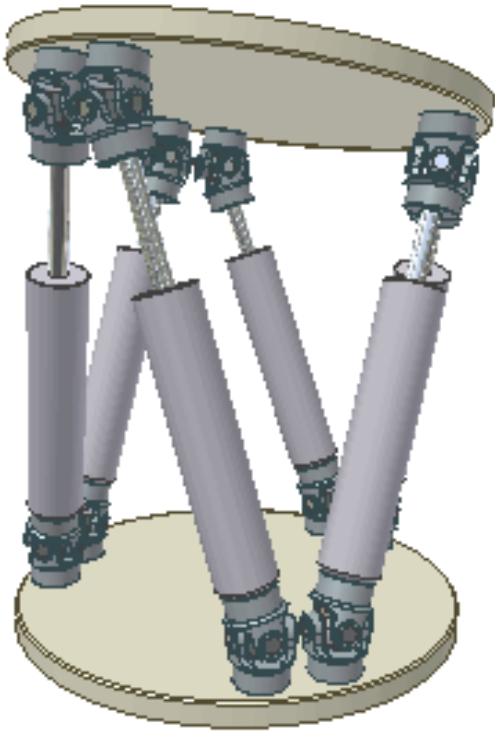


Spherical joint



Universal joint

Degrees of freedom of a robot



Grübler's formula

N = # of bodies (including ground)
 J = # of joints

$$m = \begin{cases} 3, & \text{if planar} \\ 6, & \text{if spatial} \end{cases}$$

Grübler's formula

N = # of bodies (including ground)
 J = # of joints

$$m = \begin{cases} 3, & \text{if planar} \\ 6, & \text{if spatial} \end{cases}$$

$$\text{dof} = m(N - 1) - \sum_{i=1}^J c_i$$

Grübler's formula

N = # of bodies (including ground)
 J = # of joints

$$m = \begin{cases} 3, & \text{if planar} \\ 6, & \text{if spatial} \end{cases}$$

$$\text{dof} = m(N - 1) - \sum_{i=1}^J c_i$$

Number of independent joint constraints

Grübler's formula

N = # of bodies (including ground)
 J = # of joints

$$m = \begin{cases} 3, & \text{if planar} \\ 6, & \text{if spatial} \end{cases}$$

$$\begin{aligned} \text{dof} &= m(N - 1) - \sum_{i=1}^J c_i \\ &= m(N - 1) - \sum_{i=1}^J (m - f_i) \end{aligned}$$

Grübler's formula

N = # of bodies (including ground)
 J = # of joints

$$m = \begin{cases} 3, & \text{if planar} \\ 6, & \text{if spatial} \end{cases}$$

$$\begin{aligned} \text{dof} &= m(N - 1) - \sum_{i=1}^J c_i \\ &= m(N - 1) - \sum_{i=1}^J (m - f_i) = m(N - 1 - J) + \sum_{i=1}^J f_i \end{aligned}$$

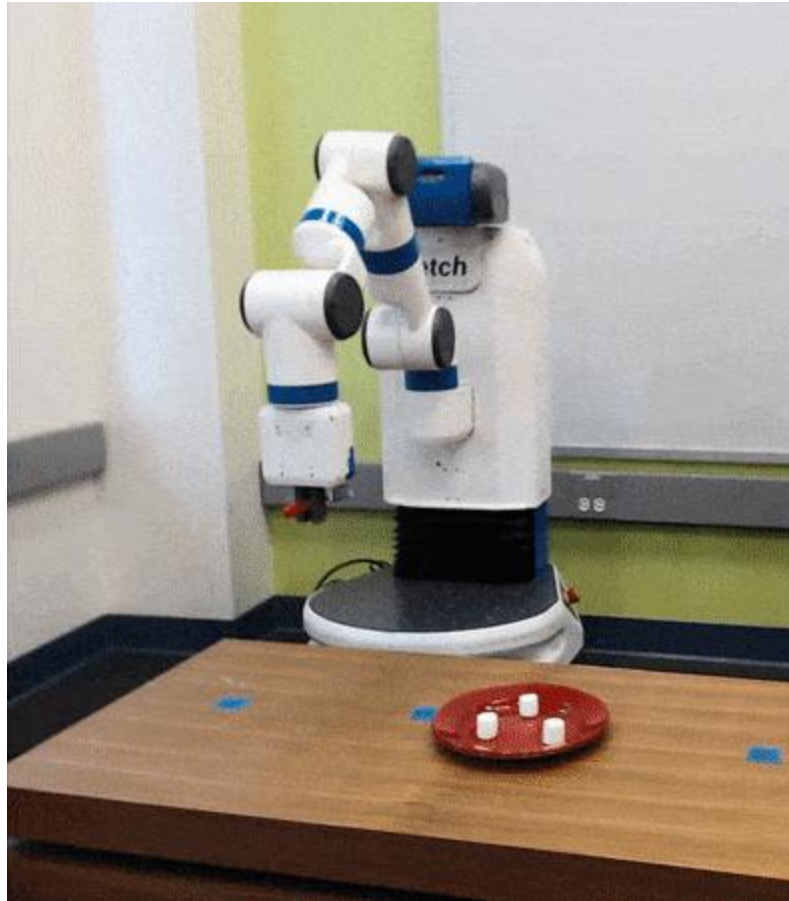
Grübler's formula

N = # of bodies (including ground)
 J = # of joints

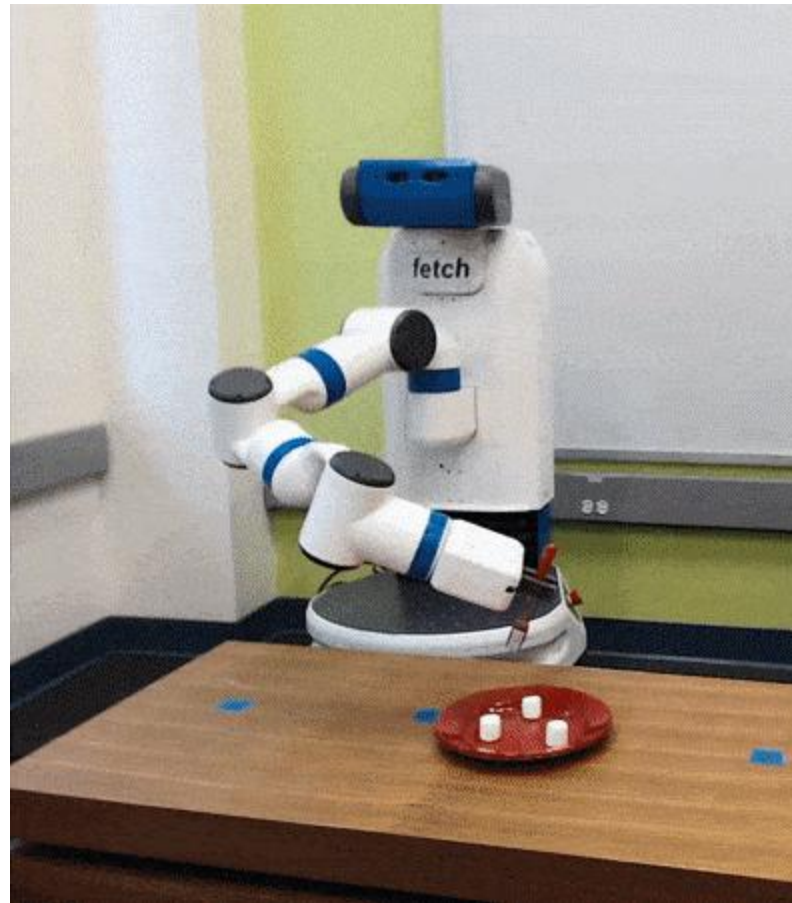
$$m = \begin{cases} 3, & \text{if planar} \\ 6, & \text{if spatial} \end{cases}$$

$$\text{dof} = m(N - 1 - J) + \sum_{i=1}^J f_i$$

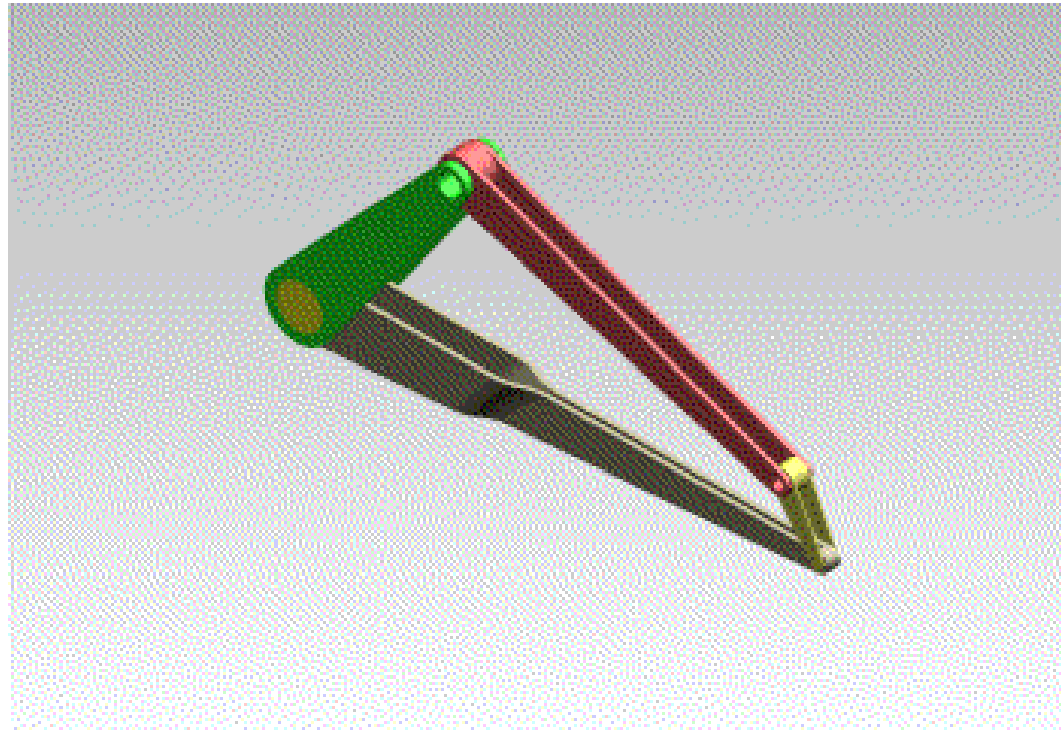
An open-chain robot arm



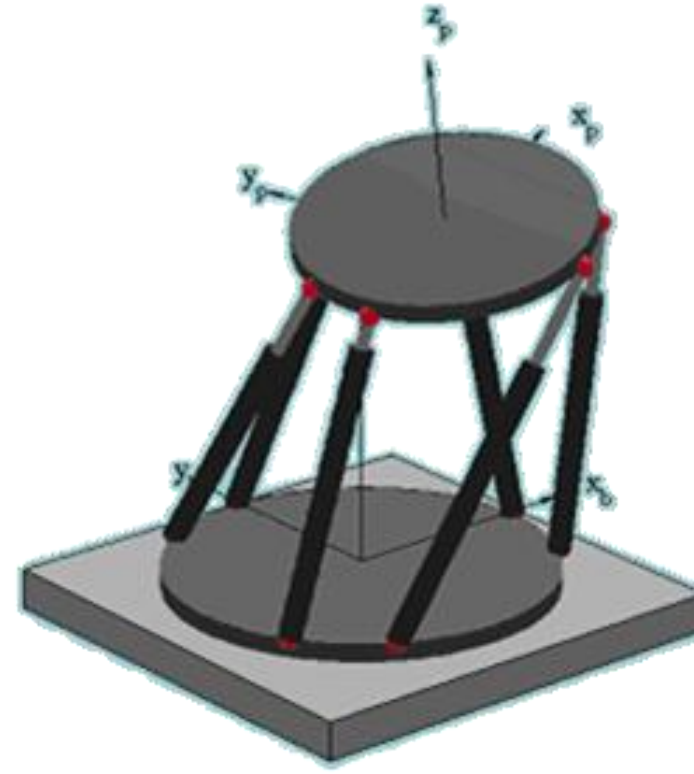
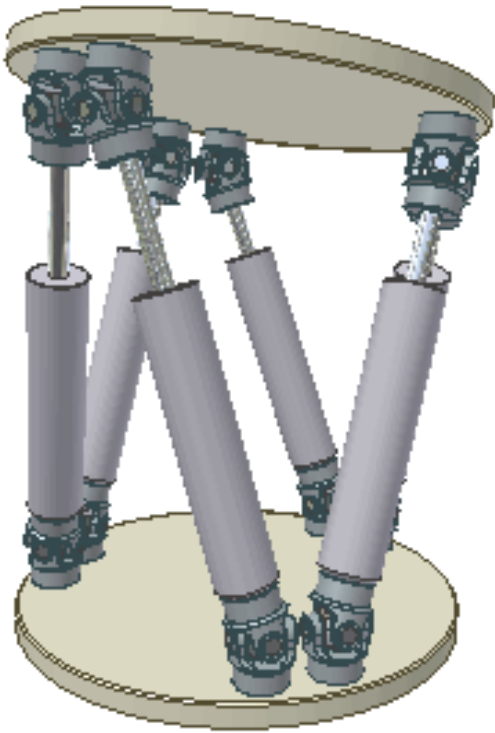
An open-chain robot arm



Four-bar closed-chain mechanism



Stewart platform



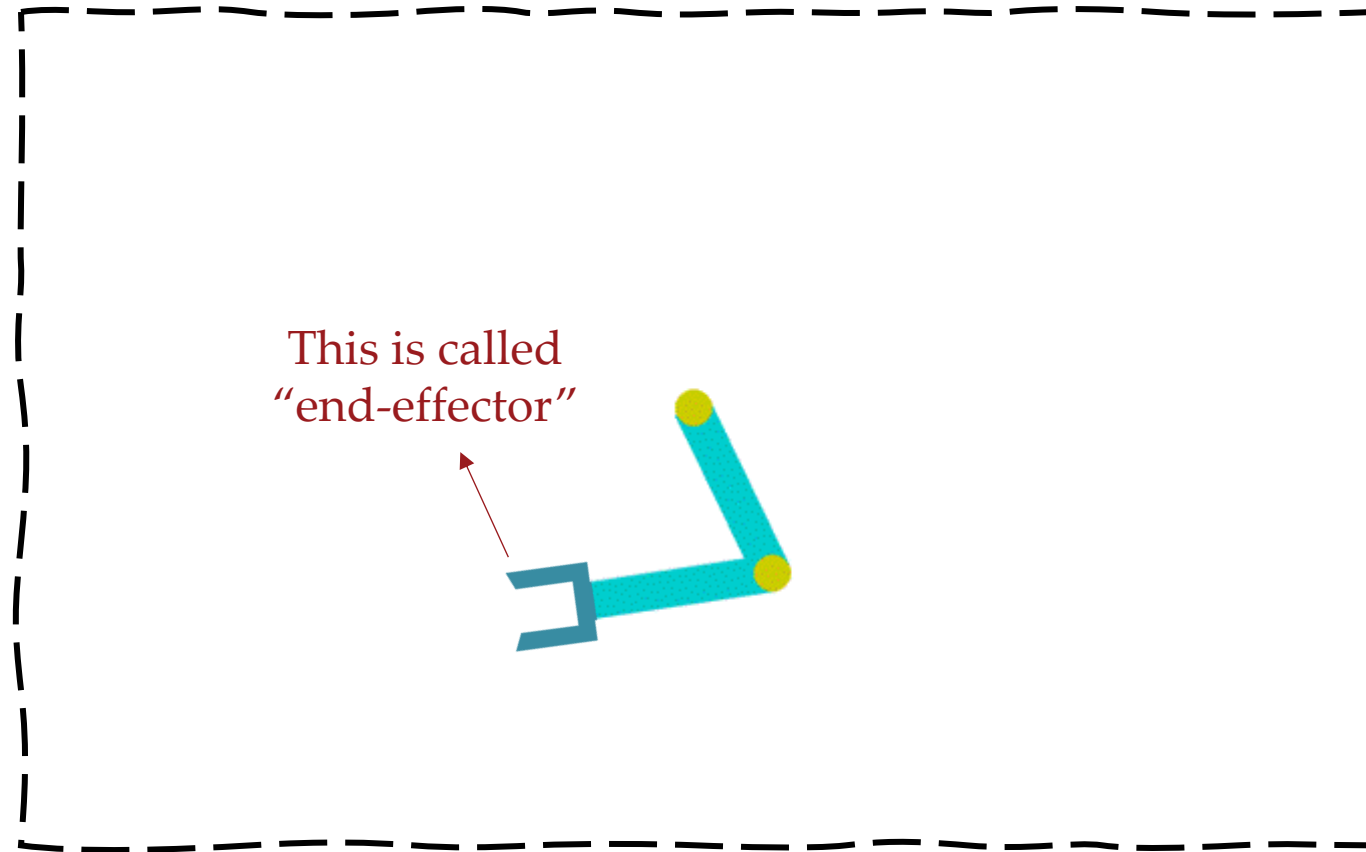


Acrobot

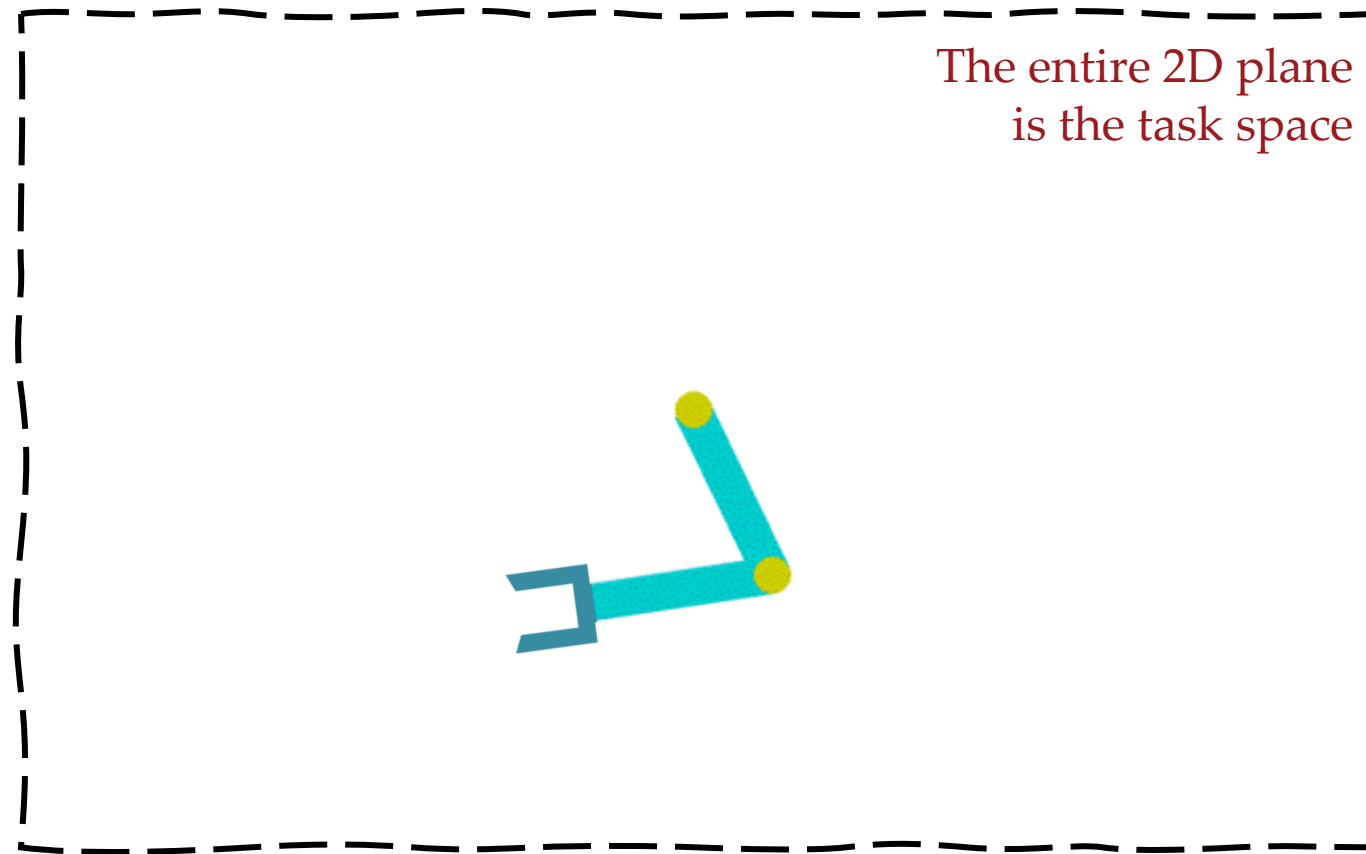


From: Gymnasium, Farama Foundation
Generalization in Reinforcement Learning: Successful Examples
Using Sparse Coarse Coding
Richard S. Sutton, NeurIPS 1995

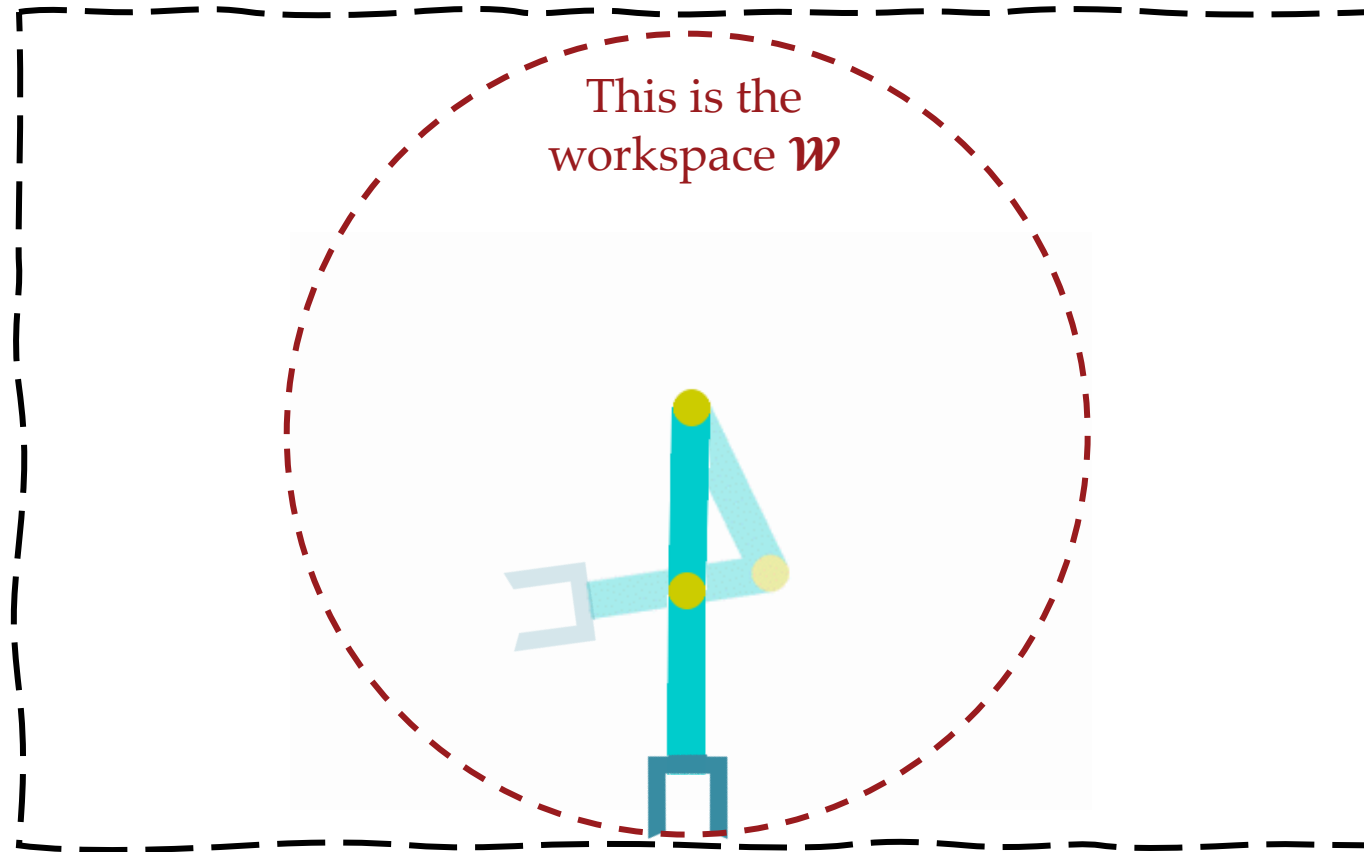
Acrobot



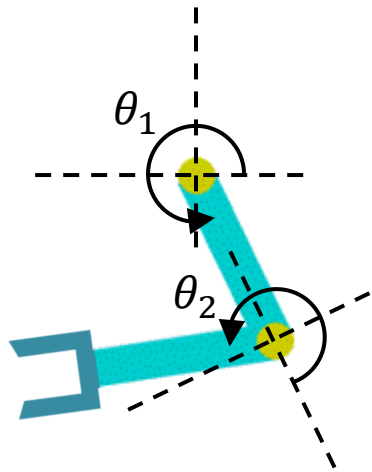
Acrobot



Acrobot



Acrobot

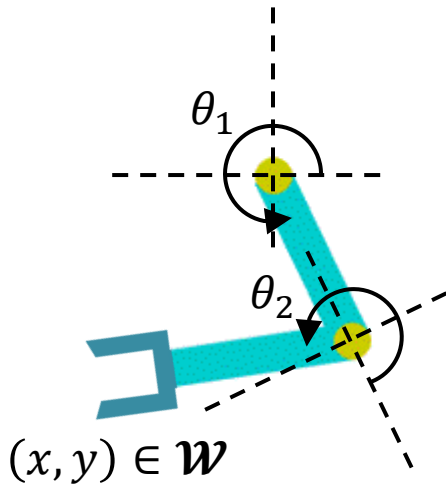


The robot's current configuration is:

$$(\theta_1, \theta_2) \in \mathcal{C}$$

This is called the
configuration space

Acrobot



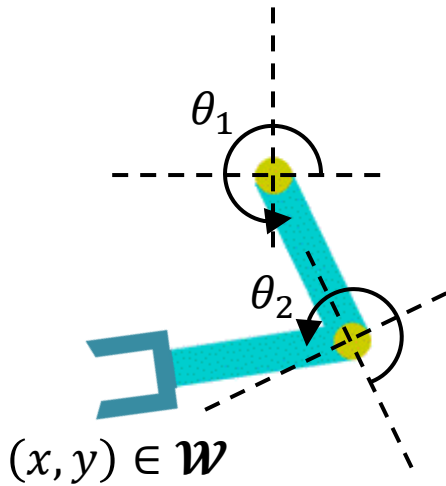
The robot's current configuration is:

$$(\theta_1, \theta_2) \in \mathcal{C}$$

Forward kinematics: $\text{FK}: \mathcal{C} \rightarrow \mathcal{W}$

$$\text{FK}((\theta_1, \theta_2)) = (x, y)$$

Acrobot



The robot's current configuration is:

$$(\theta_1, \theta_2) \in \mathcal{C}$$

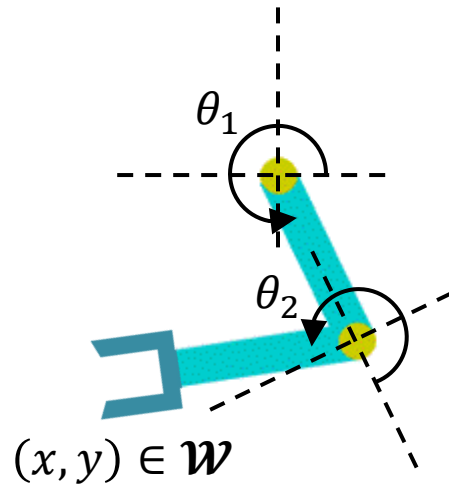
Forward kinematics: FK: $\mathcal{C} \rightarrow \mathcal{W}$

$$\text{FK}((\theta_1, \theta_2)) = (x, y)$$

Inverse kinematics: IK: $\mathcal{W} \rightarrow \mathcal{C}$

$$\text{IK}((x, y)) = (\theta_1, \theta_2)$$

Acrobot



The robot's current configuration is:

$$(\theta_1, \theta_2) \in \mathcal{C}$$

Forward kinematics: FK: $\mathcal{C} \rightarrow \mathcal{W}$

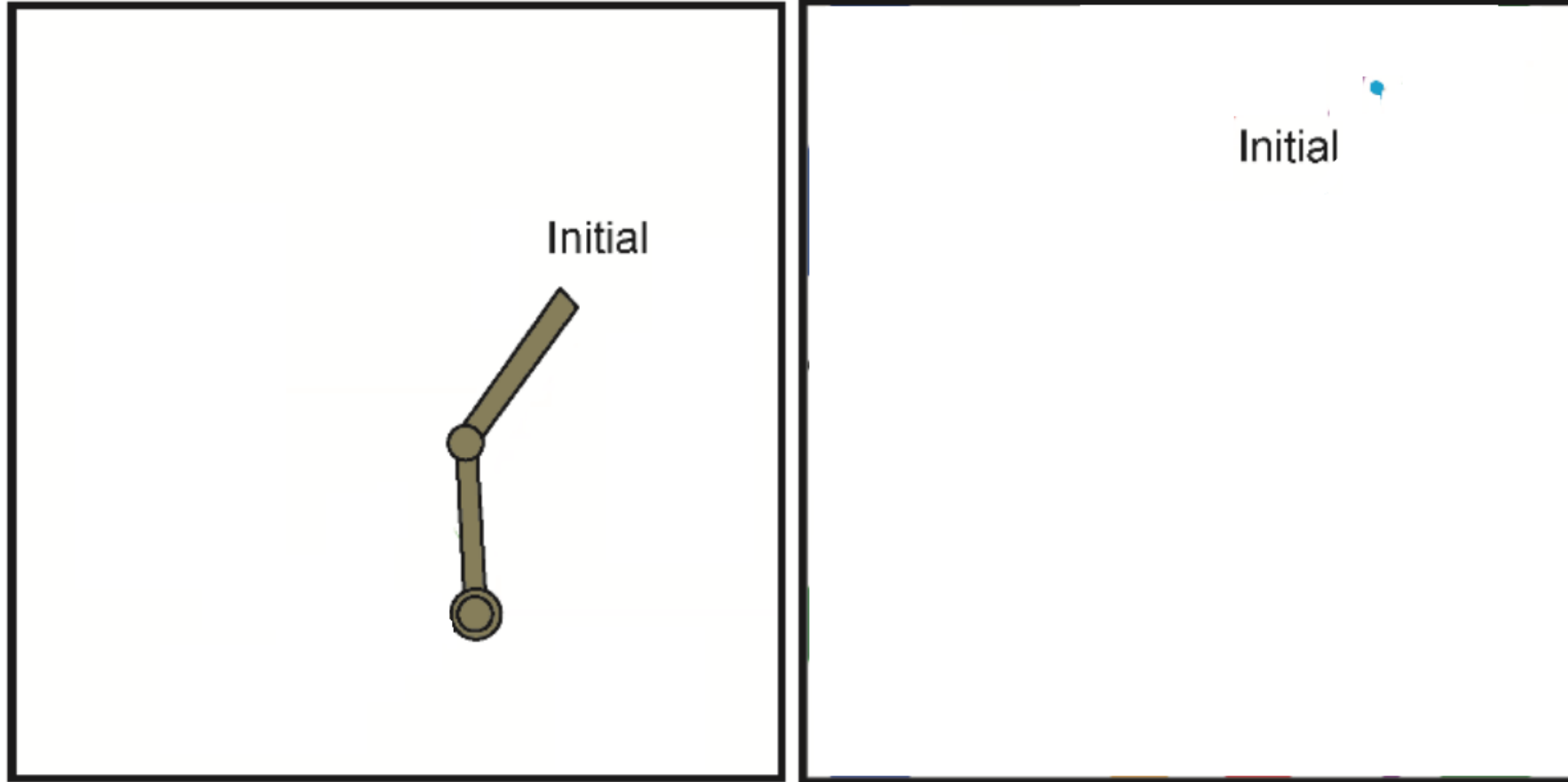
$$\text{FK}((\theta_1, \theta_2)) = (x, y)$$

Inverse kinematics: IK: $\mathcal{W} \rightarrow \mathcal{C}$

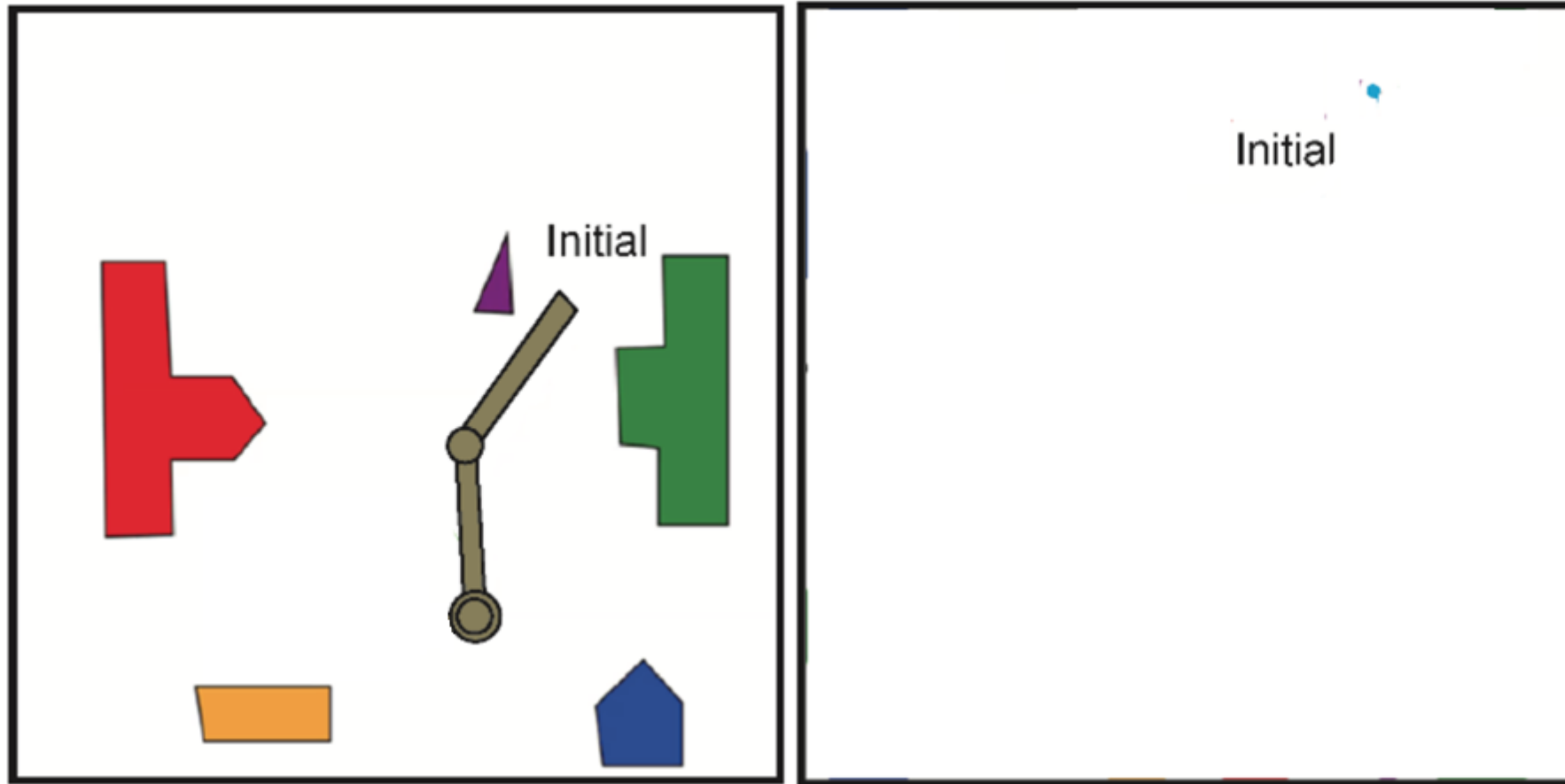
$$\text{IK}((x, y)) = (\theta_1, \theta_2)$$

This is often not a proper function.
Because many configurations may
lead to the same end-effector pose.

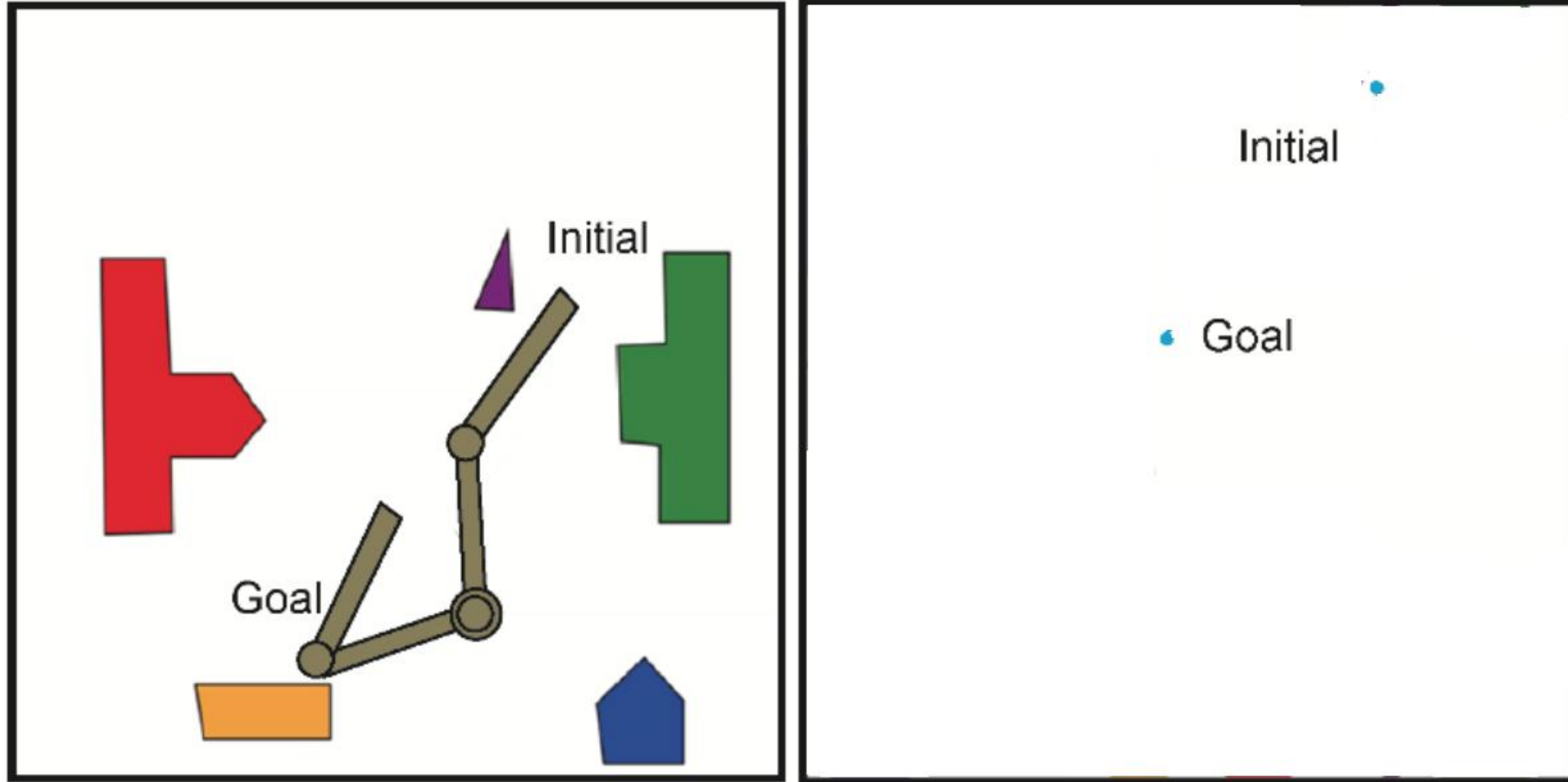
Okay, but why?



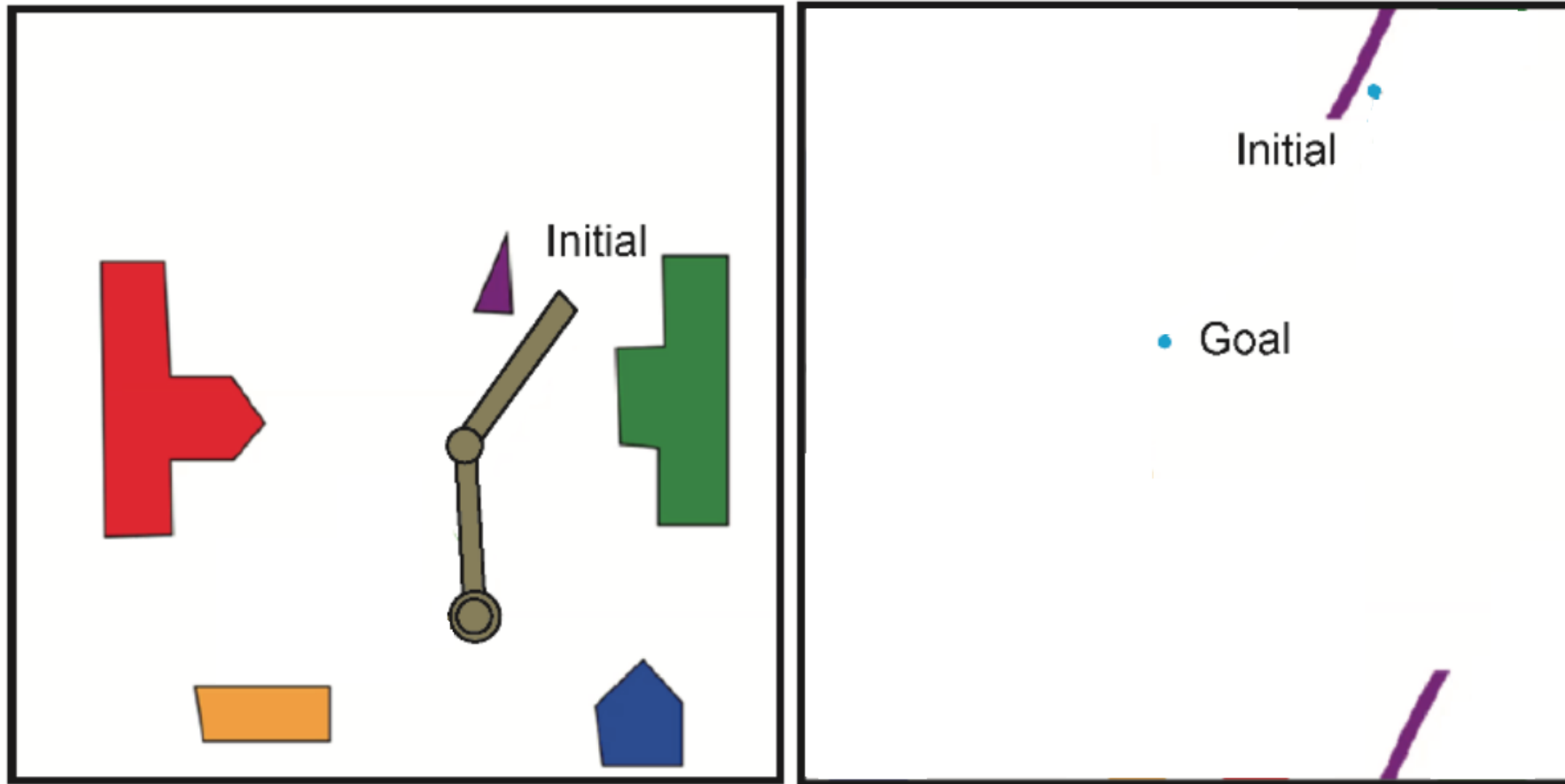
Okay, but why?



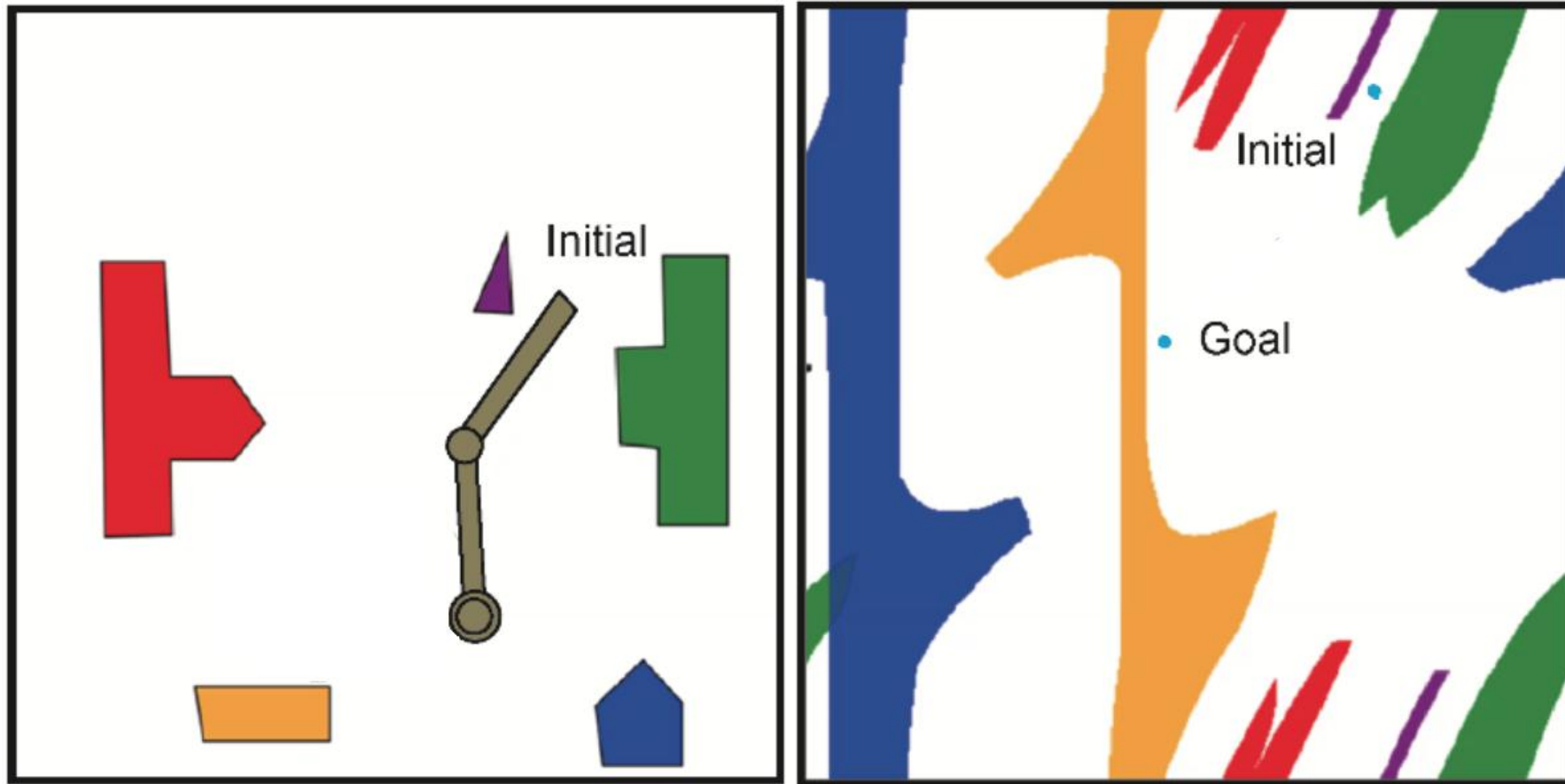
Okay, but why?



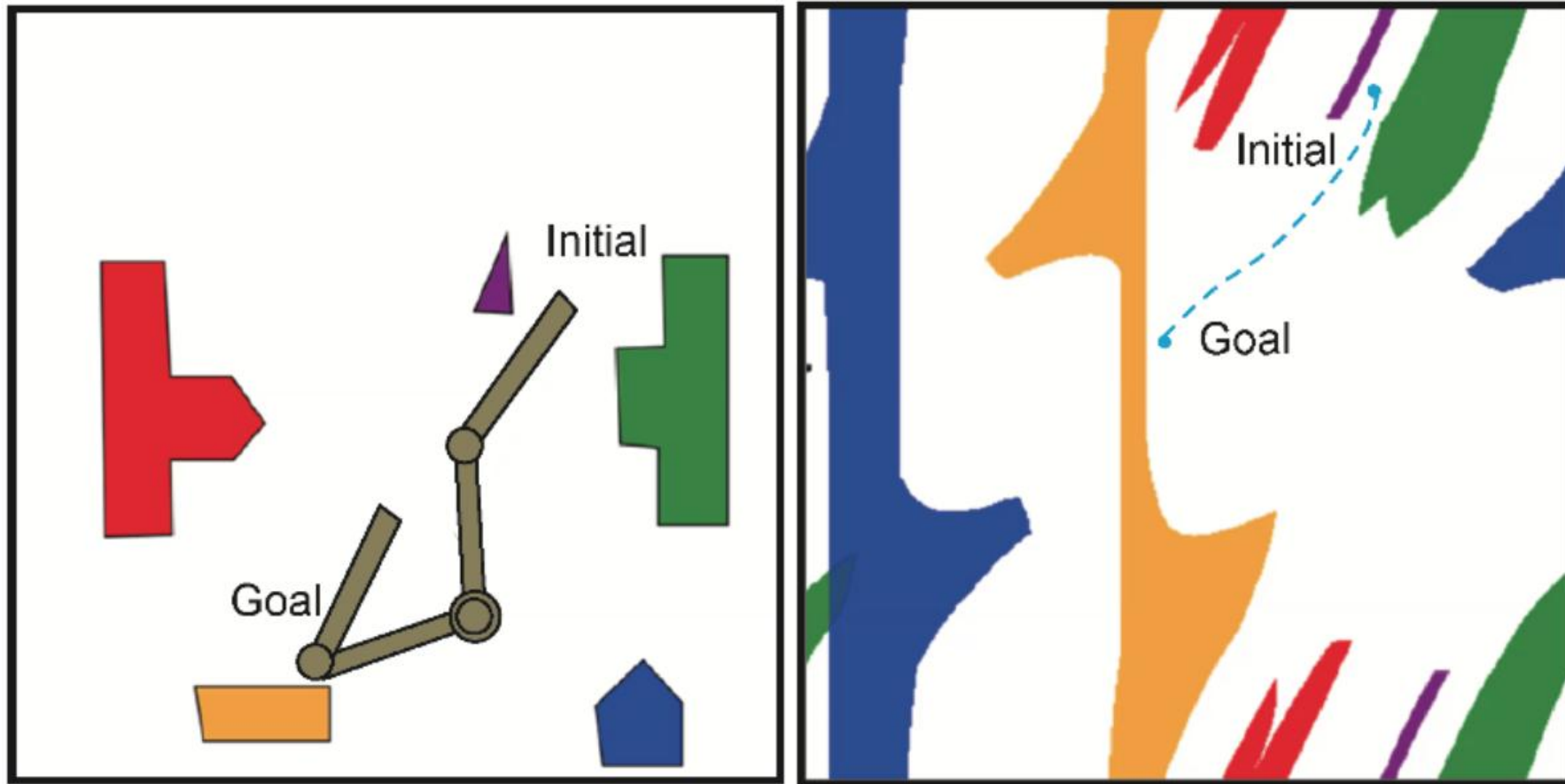
Okay, but why?



Okay, but why?



Okay, but why?



Today...

- General course information
- Basics of robotics
- Fundamentals of machine learning

Machine learning

Supervised learning

Given $\{(x^i, y^i)\}_{i=1}^n$, find a function

$$f(x) = y$$



(classification, regression)

Unsupervised learning

Machine learning

Supervised learning

Given $\{(x^i, y^i)\}_{i=1}^n$, find a function

$$f(x) = y$$

data point
predictor

...

value
label

...

(classification, regression)

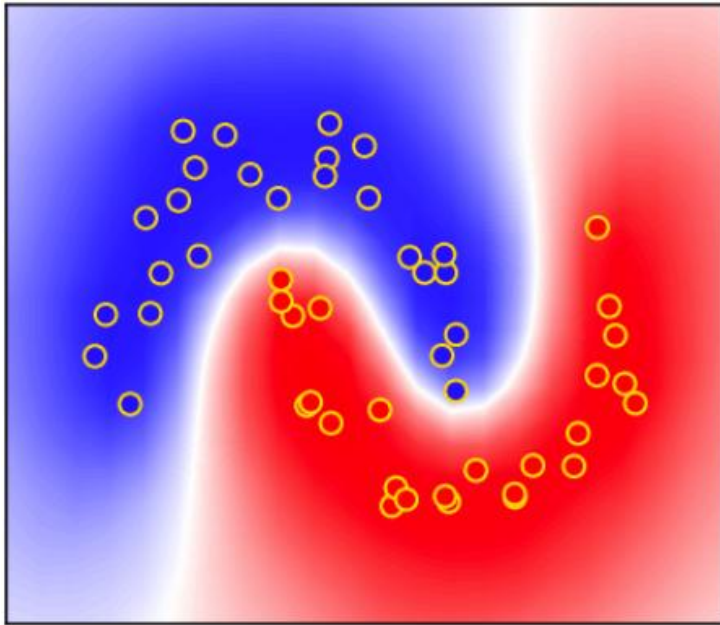
Unsupervised learning

Given $\{x^i\}_{i=1}^n$, find patterns

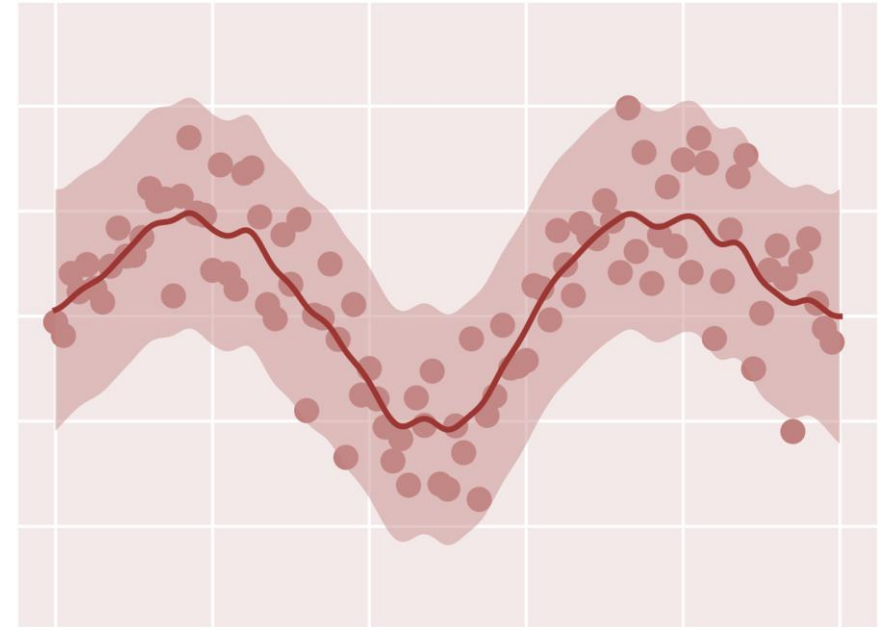
(clustering, compression,
dimensionality reduction)

Supervised learning

Classification



Regression



Learning models

- Parametric models:

$$y = f_{\theta}(x)$$

Examples: naïve Bayes, logistic regression, neural networks

Learning models

- Parametric models:

$$y = f_{\theta}(x)$$

Examples: naïve Bayes, logistic regression, neural networks

- Non-parametric models:

$$y = f(x; D)$$

dataset



Examples: K-nearest neighbors, Gaussian process regression

Loss functions

A loss function evaluates the quality of fit in $f(x) \approx y$ or the quality of patterns in an unsupervised learning problem.

Examples:

ℓ^2 loss:
$$L(\theta) = \sum_{(x^i, y^i) \in D} (y^i - f_{\theta}(x^i))^2$$

Cross-entropy loss:
$$L(\theta) = - \sum_{(x^i, y^i) \in D} (y^i)^{\top} \log f_{\theta}(x^i)$$

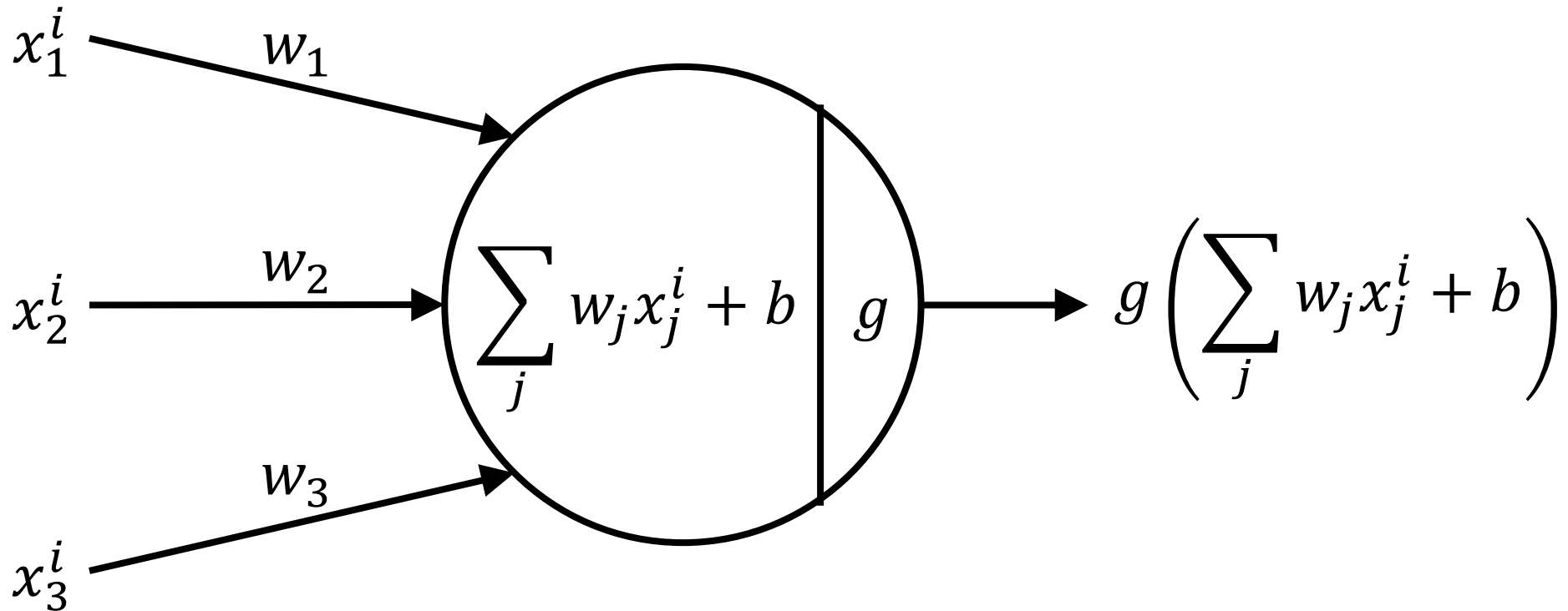
Minimizing the loss

- Analytical solution
 - Use exact methods to find $\theta^* = \arg \min_{\theta} L(\theta)$
 - Occasionally possible, e.g., linear regression
- Numerical optimization
 - Numerically minimize $L(\theta)$, e.g., gradient descent by computing $\nabla L(\theta)$
 - Much more common in robot learning research
 - Stochastic optimization is often necessary for efficiency

Neural networks

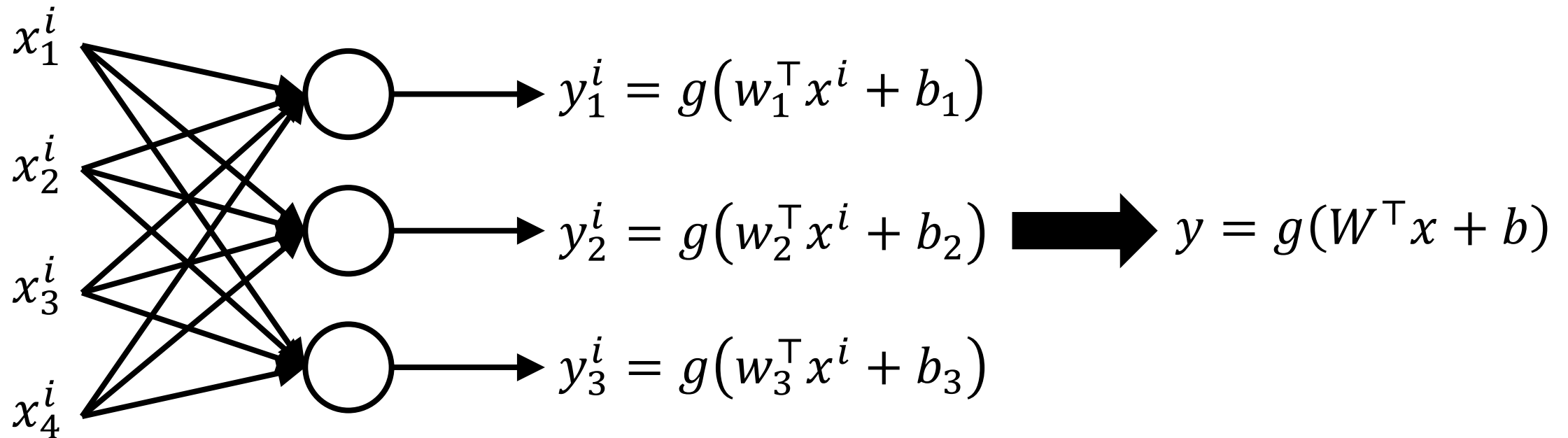
This is not the first model taught in a machine learning class. But we will almost never use other models.

1. A perceptron



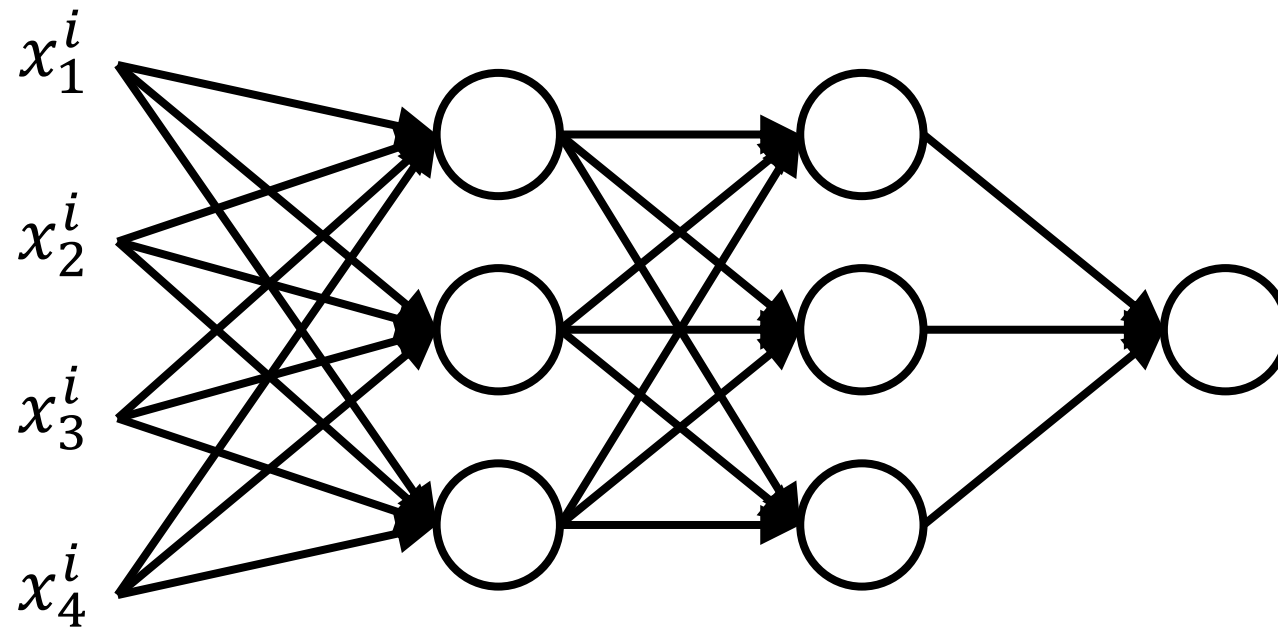
Neural networks

2. A single layer neural network

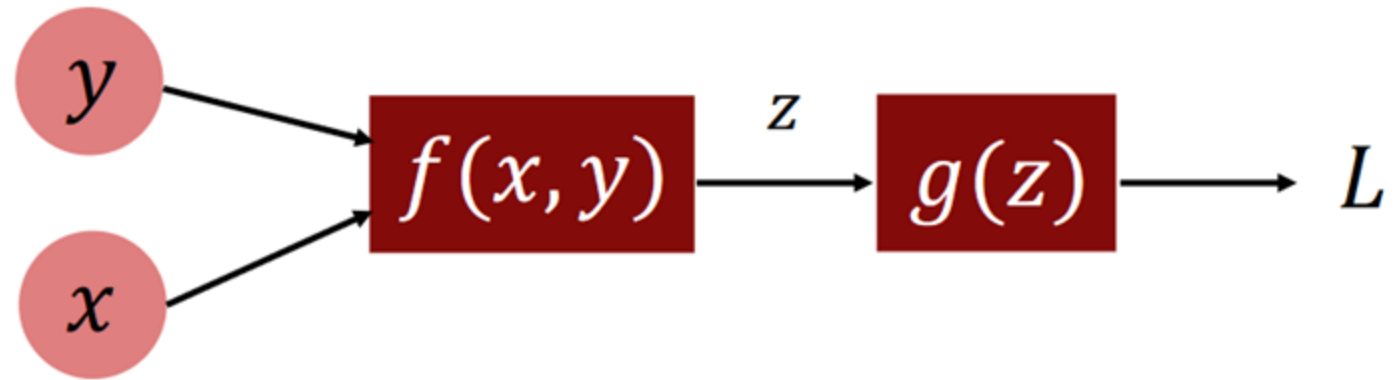


Neural networks

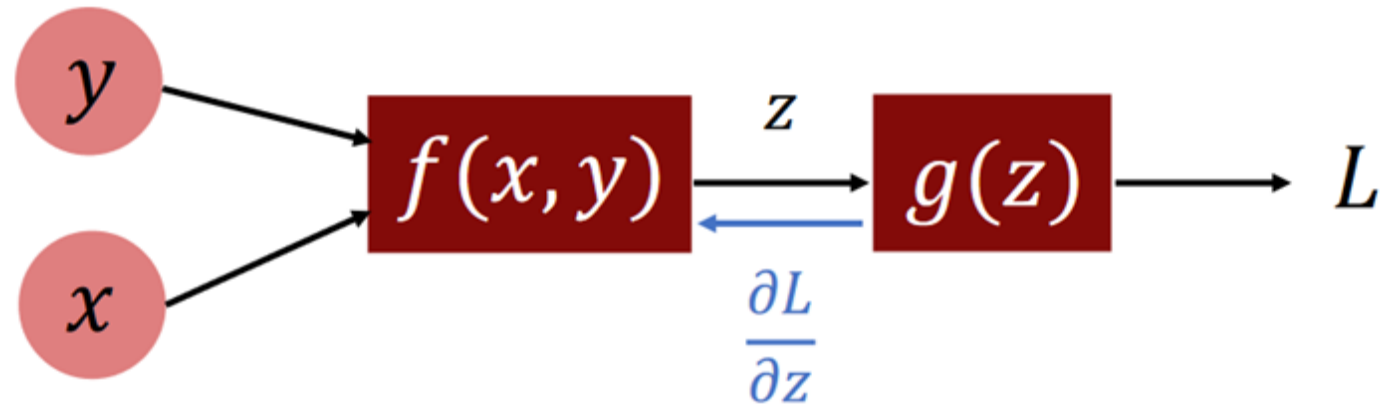
3. A deep neural network



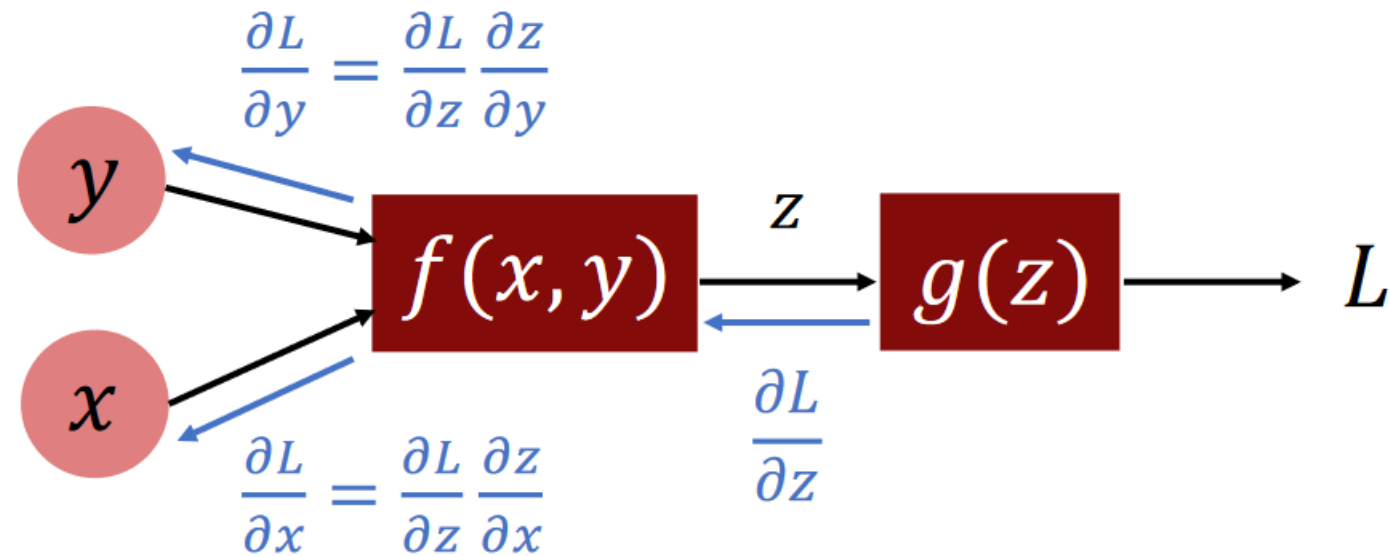
Backpropagation



Backpropagation

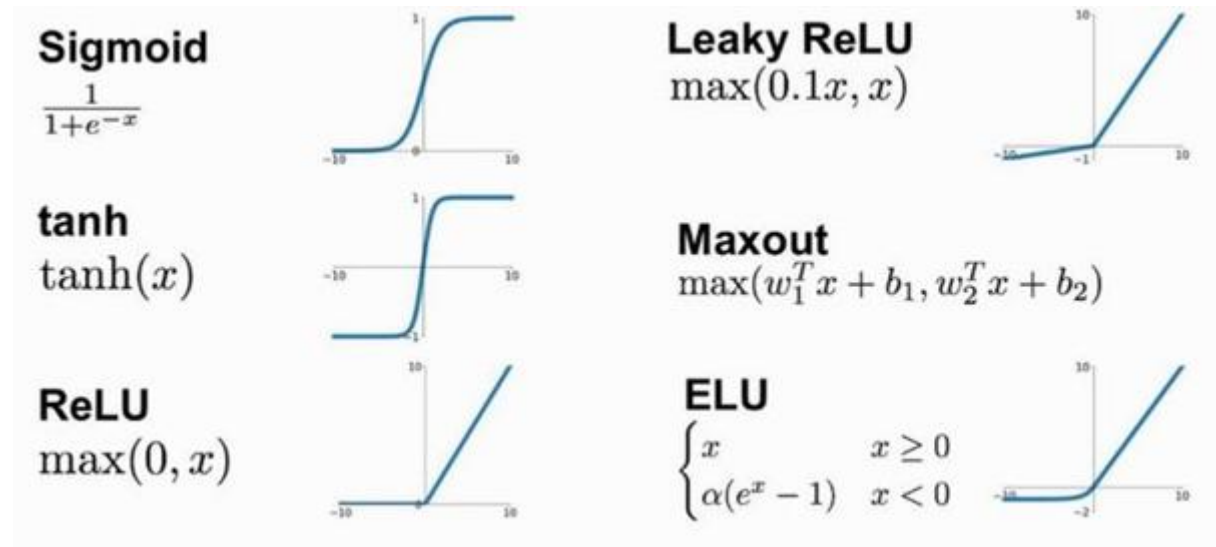


Backpropagation



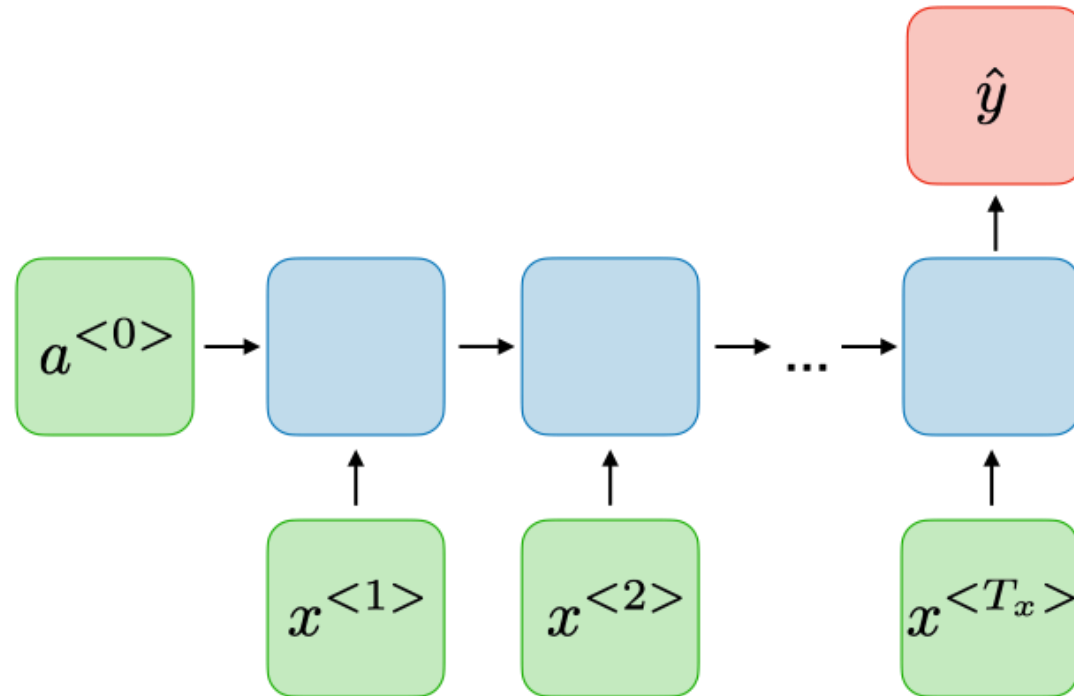
Activation functions

g should not be a linear function.



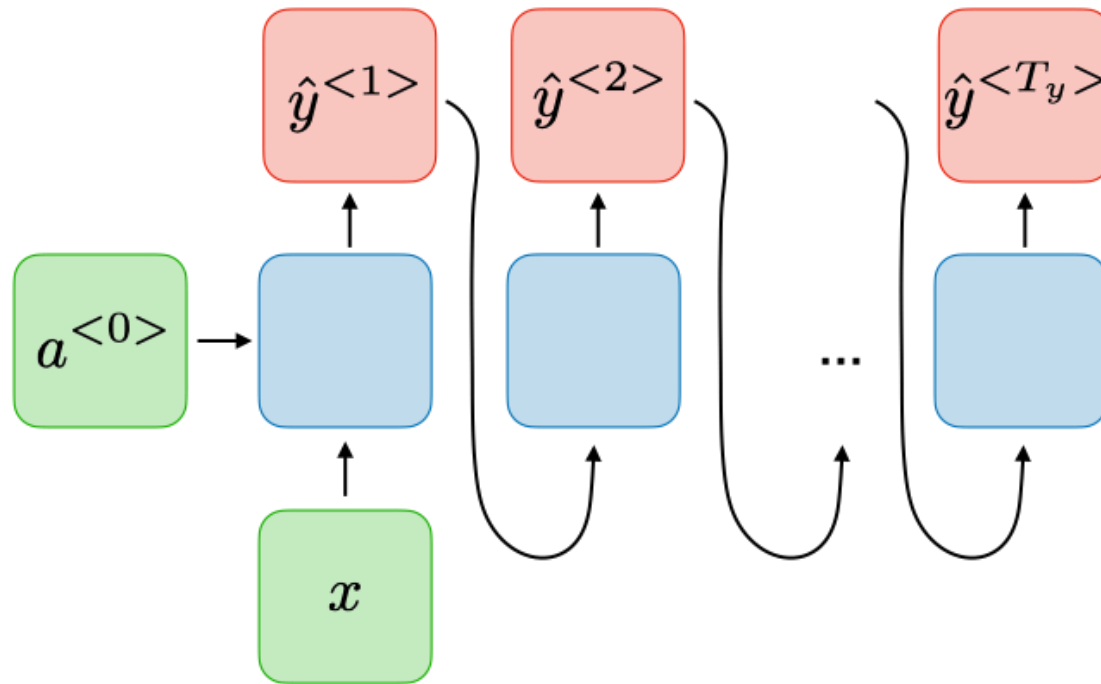
Recurrent neural networks (RNN)

Many-to-one



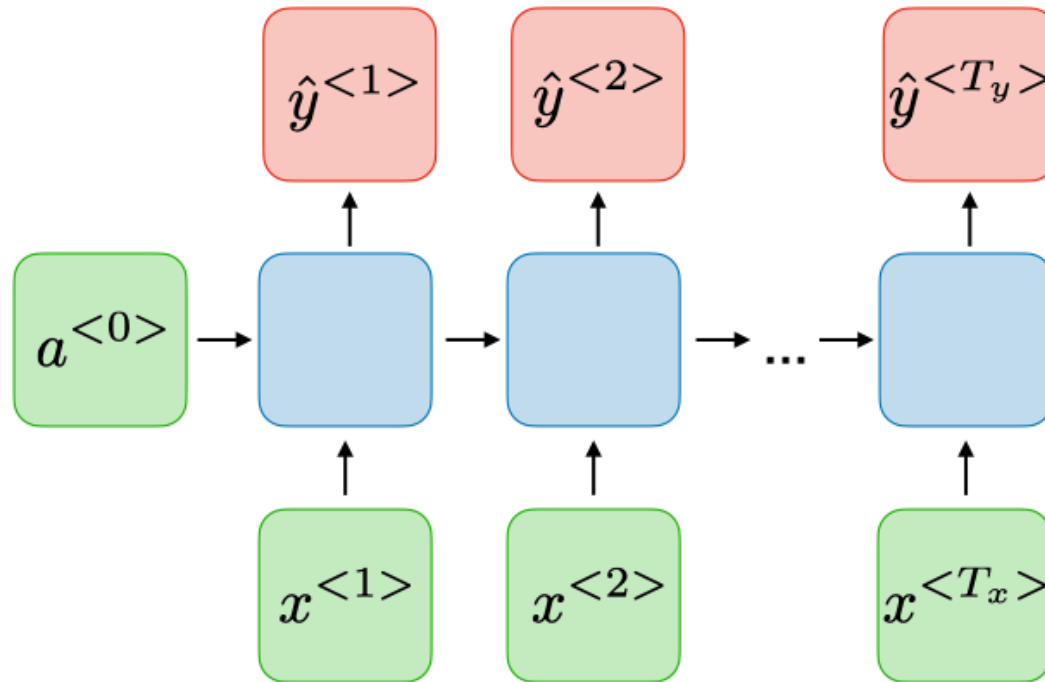
Recurrent neural networks (RNN)

One-to-many



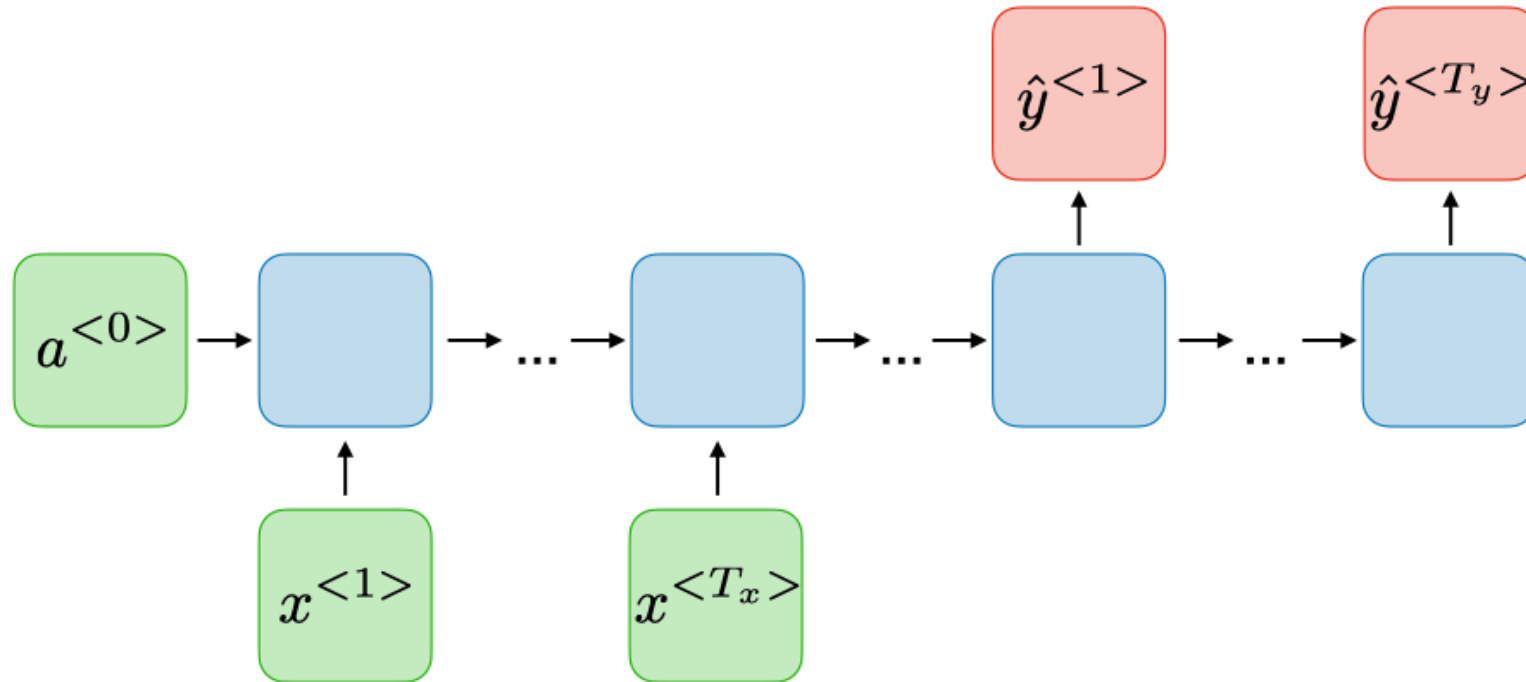
Recurrent neural networks (RNN)

Many-to-many

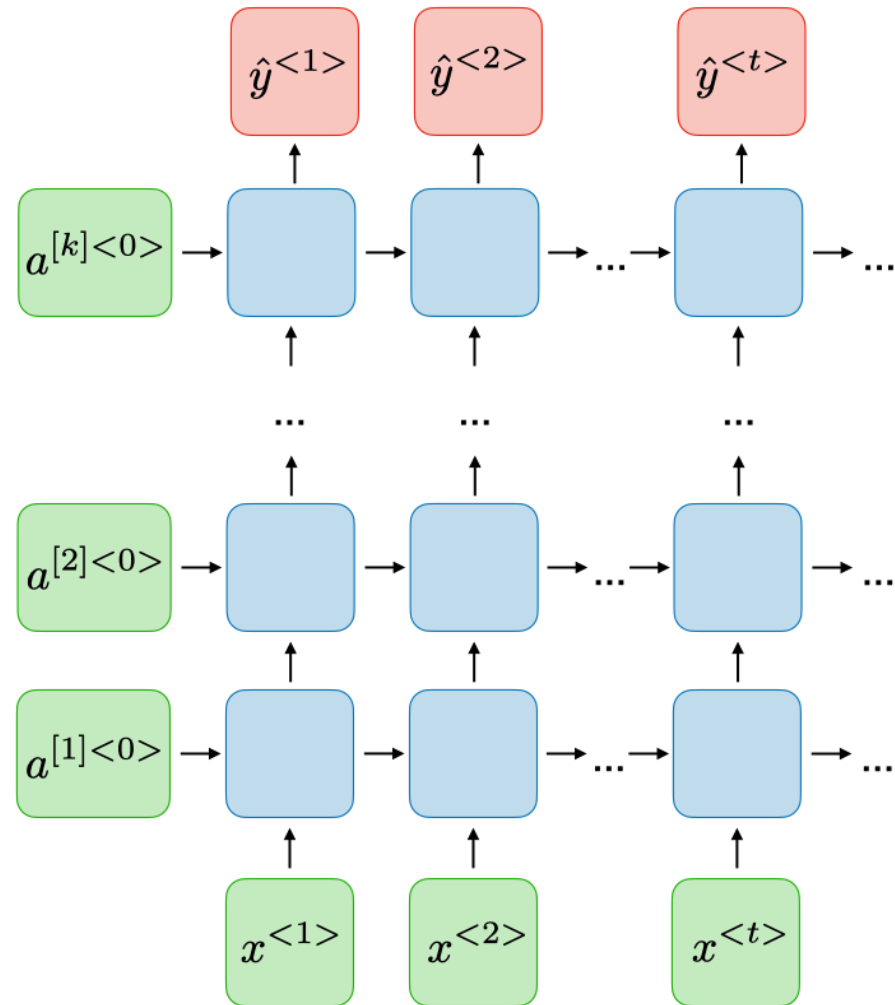


Recurrent neural networks (RNN)

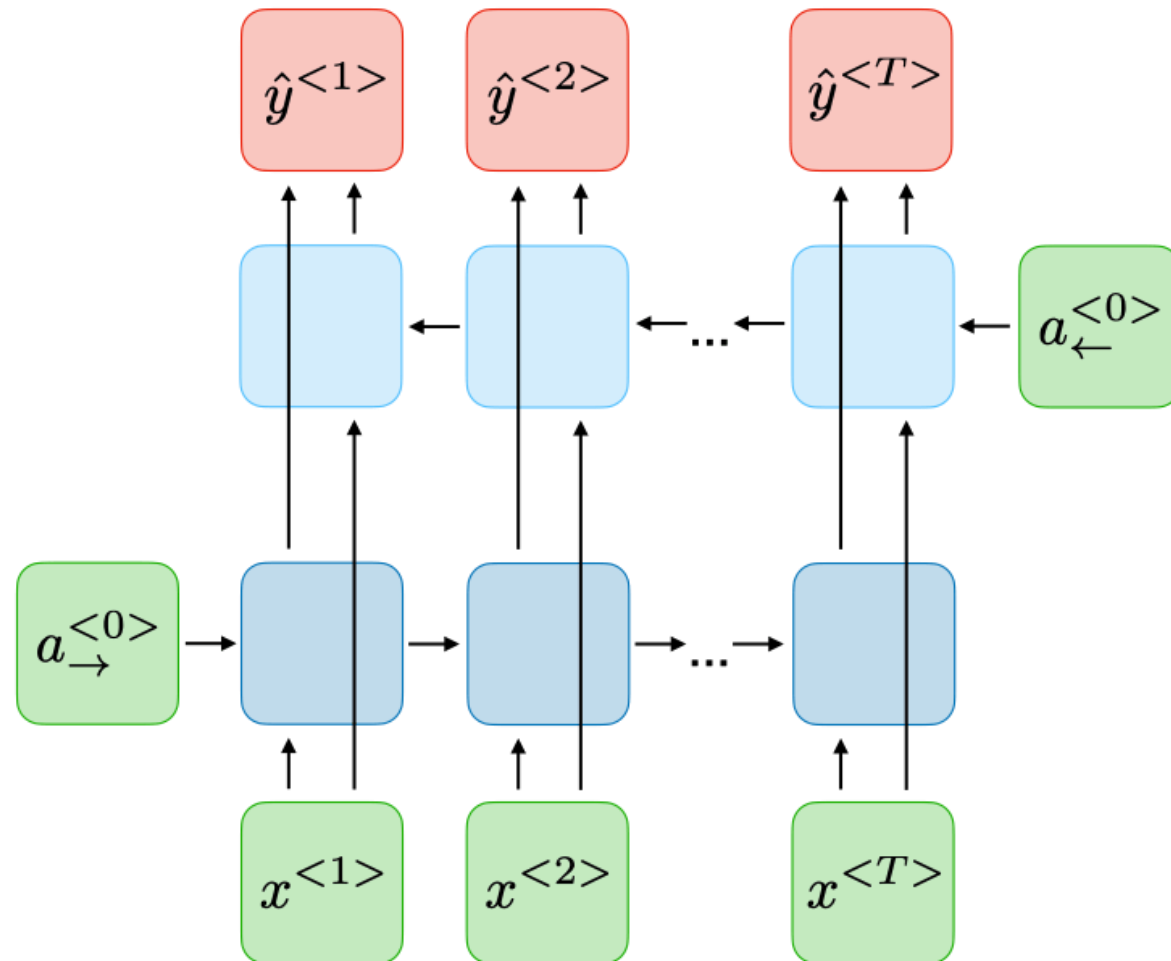
Many-to-many



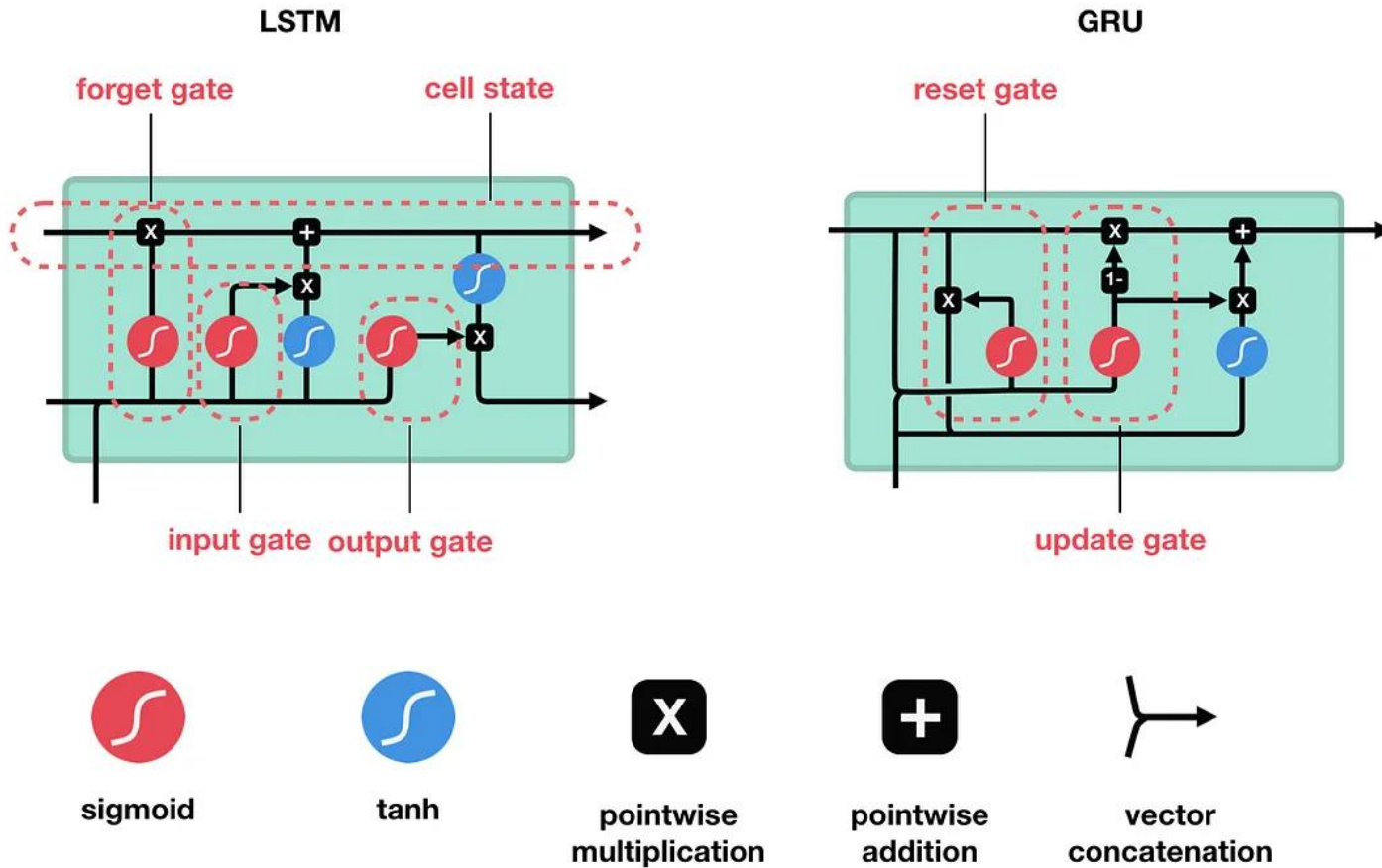
Deep RNNs



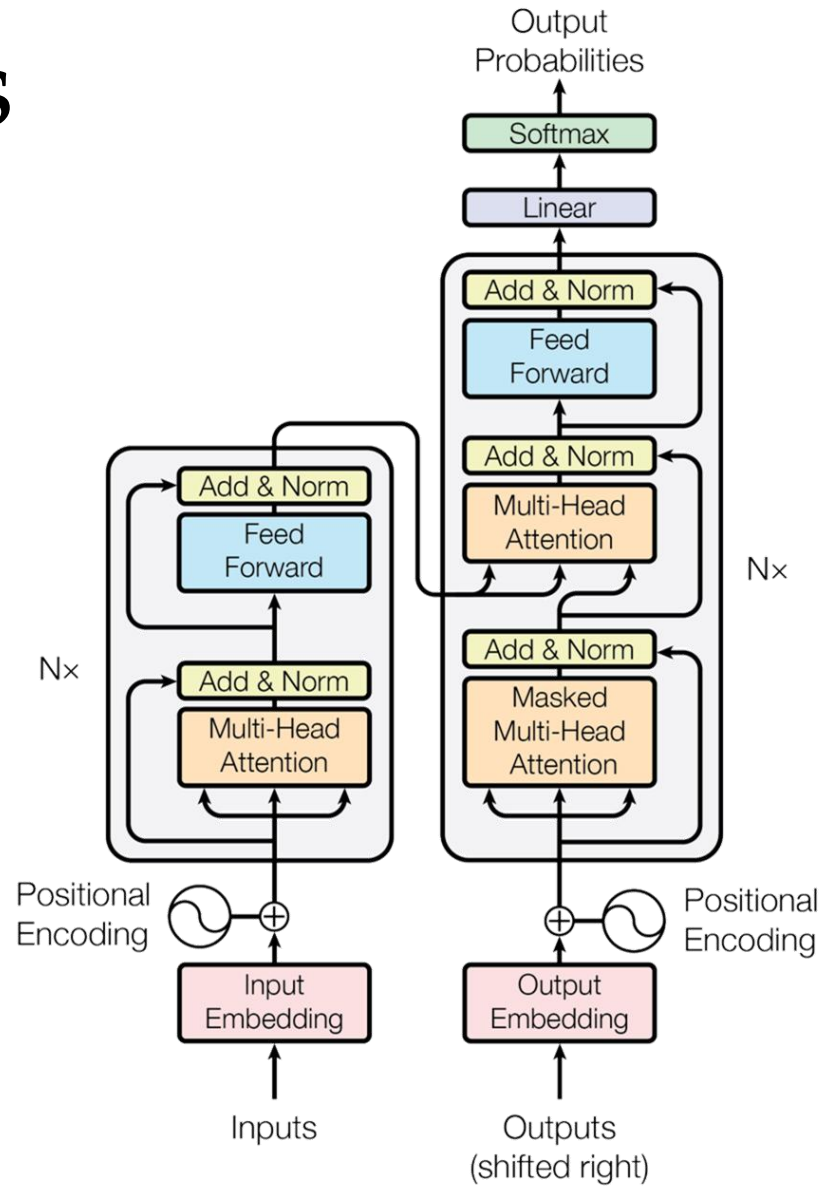
Bidirectional RNNs



LSTMs and GRUs



Transformers



Today...

- General course information
- Basics of robotics
- Fundamentals of machine learning

Until next week...

Homework assignments will include programming with a machine learning library: PyTorch.

There are many online PyTorch tutorials. For what we covered today, check out:

- https://pytorch.org/tutorials/beginner/blitz/tensor_tutorial.html
- https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html

Next time...

- Basics of computer vision for robotics
- Representation learning